
MASTER THESIS

FROM POTENTIAL ENERGY SURFACE
TO
GAS ADSORPTION
VIA
DEEP LEARNING

Defended on

FEBRUARY 21, 2024

ANTONIOS P. SARIKAS

 chempi60@edu.chemistry.uoc.gr 

Supervised by

GEORGE E. FROUDAKIS

MATERIALS MODELING



DESIGN GROUP

DEPARTMENT OF CHEMISTRY



UNIVERSITY OF CRETE



To my father

**Special thanks to my family, Marina, Stella, Angelos and Panagiotis,
for their continued love and support.**

Contents

List of Figures	6
List of Algorithms	7
1 Introduction	10
1.1 Applications of Reticular Chemistry	10
1.2 The Problem	11
1.3 Literature Review	12
1.4 Thesis Statement	14
2 Theoretical Background	15
2.1 Machine Learning Preliminaries	15
2.1.1 Learning paradigms	16
2.1.2 Formulating the problem of supervised learning	17
2.1.3 Components of a learning algorithm	18
2.1.4 Performance, complexity and experience	20
2.2 Fundamentals of Deep Learning	23
2.2.1 Neural networks	25
2.2.2 Regularizing neural networks	29
2.2.3 Convolutional neural networks	30
2.2.4 Training neural networks	34
3 Methodology	41
3.1 Datasets	41
3.1.1 MOFs dataset	41
3.1.2 COFs dataset	41
3.2 Voxelized PES	42
3.3 Machine Learning Details	43
3.3.1 CNN architecture	43
3.3.2 Preprocessing & CNN training details	43
3.3.3 Data augmentation	44
4 Results & Discussion	46
4.1 Visualizing RetNet	46
4.2 Learning Curves & Parity Plots	48
4.3 Discussion	49
Bibliography	52

Index	59
Acronyms	63
A $\text{MOX}_{\epsilon\lambda}$	65
B RetNet	76
C Training RetNet	85
D Training Random Forest	88

List of Figures

1.1	Unit cell structure of IRMOF-1.	11
1.2	Material space of MOFs.	12
1.3	Generalized framework to predict gas adsorption properties.	14
2.1	Main tasks of supervised learning.	17
2.2	The bias-variance trade-off.	22
2.3	Relation between performance and experience.	23
2.4	The perceptron.	24
2.5	Examples of activation functions.	25
2.6	The multilayer perceptron.	26
2.7	Examples of graphs	26
2.8	Solving the XOR problem.	28
2.9	Convolution operation.	31
2.10	Neurons' arrangement in convolutional and dense layers.	32
2.11	Max pooling operation.	33
2.12	Illustration of back-propagation.	36
2.13	Machine learning $\neq$ optimization.	37
2.14	Variants of gradient descent.	38
3.1	Workflow to construct the voxelized PES.	42
3.2	Geometric transformations for data augmentation.	44
3.3	Effect of data augmentation.	45
4.1	RetNet architecture.	47
4.2	Fingerprints extracted from RetNet.	48
4.3	Learning curves.	49
4.4	Parity plots.	50

List of Algorithms

1	Gradient descent	20
2	Batch normalization	35
3	Back-propagation	37
4	Batch, mini-batch and stochastic gradient descent	39
5	Momentum	39
6	Adam	40

TRILATERAL



COMMISSION

GEORGE E. FROUDAKIS

 frudakis@chemistry.uoc.gr 

Professor of Computational Chemistry

PANTELIS N. TRIKALITIS

 ptrikal@uoc.gr 

Professor of Inorganic Chemistry

CONSTANTINOS G. NEOCHORITIS

 kneochor@uoc.gr 

Assistant Professor of Organic Chemistry

HERAKLION, 2024



METAL-ORGANIC FRAMEWORKS, or in short MOFs, thanks to their ultra high porosity and surface area, are deemed as prominent candidates for applications involving gas adsorption. However, their intrinsic combinatorial nature translates to a practically infinite material space, rendering the identification of novel materials with traditional methods cumbersome. Over the last years, machine learning approaches based on predictive models have been developed, allowing researchers to rapidly screen large databases of MOFs. The quality of these models is highly dependent on the mathematical representation of a material, thus necessitating the use of informative inputs. In this thesis, we propose a generalized framework for predicting gas adsorption properties, using as one and only input the potential energy surface. We treat the latter as a 3D energy image and then pass it through a 3D convolutional neural network, known for its ability to process image-like data. The proposed pipeline is applied in MOFs for predicting CO₂ uptake. The resulting model outperforms both in terms of accuracy and data efficiency a conventional one built upon textual properties. Additionally, we demonstrate the transferability of the approach to other host-guest systems, by examining CH₄ uptake in covalent organic frameworks. The performance and generality of the proposed approach along with the fast input calculation thanks to parallelization, renders it suitable for large scale screening. Finally, discussion for improving and extending the suggested scheme is provided.

Chapter I

Introduction



RETICULAR CHEMISTRY, a field that bridges inorganic and organic chemistry (Yaghi 2020), has emerged from a simple albeit powerful idea: *combining molecular building blocks to form extended crystalline structures* (Yaghi 2019). It all started in 1990s, with the advent of metal-organic frameworks (MOFs), the first “offspring” of reticular chemistry. MOFs, a class of nanoporous materials *composed of metal ions or clusters coordinated to organic ligands aka organic linkers*, possess extraordinary properties, such as ultrahigh porosity and huge surface areas (Farha et al. 2012). To get a sense of how extraordinary these materials are, it is suffice to say that *one gram of such a material can have a surface area as large as a soccer field*. The fact that reticular materials are “brought to life” by combining simple building blocks, allows chemists and material scientists to design materials in a judicious manner. The epitome of design in reticular chemistry is found in the synthesis of a zirconium-based MOF (Trikalitis et al. 2016), incorporating the polybenzene network or “cubic graphite” structure, predicted about 70 years ago.

1.1 Applications of Reticular Chemistry

Owing to their aforescribed properties along with their extremely tunable and modular nature, MOFs have been considered prominent solutions for gas-adsorption related problems (Y. Li et al. 2007; Jiang et al. 2022). MOFs find application in fields such as gas storage and separation, catalysis and drug delivery, just to name a few.

Carbon capture is a prime example (An et al. 2009; Sumida et al. 2011; Qazvini et al. 2021), where MOF-based sorbents have been deemed as green, low-cost and energy-efficient solutions. These materials provide versatile solutions to carbon capture, spanning various phases of the capture process, with direct air capture (DAC) being a noteworthy example. DAC includes chemical or physical methods for extracting carbon dioxide directly from the ambient air, with MOF-powered DAC showing great potential as a green and sustainable strategy for reducing carbon dioxide levels, contributing to the combating of climate change (Bose et al. 2023).

Hydrogen storage is one of the greatest challenges of hydrogen economy, currently inhibiting the transition from fossil fuels to hydrogen. Fortunately, characteristics of MOF adsorbents such as fast adsorption/desorption kinetics, low operating pressures and high hydrogen capacities, render them as promising answers to the aforementioned challenge (Suh et al. 2011; Suresh et al. 2021).

Methane is an attractive fuel for vehicular applications, being a relatively clean-burning fuel compared to gasoline. **Methane storage** in sorbents known as adsorbed natural gas (ANG) exhibit advantages over compressed natural gas (CNG) and liquefied natural gas (LNG), both in terms of energy-efficiency and vehicular safety. MOFs (S. Ma et al. 2007; Spanopoulos et al. 2016; Tsangarakis et al.

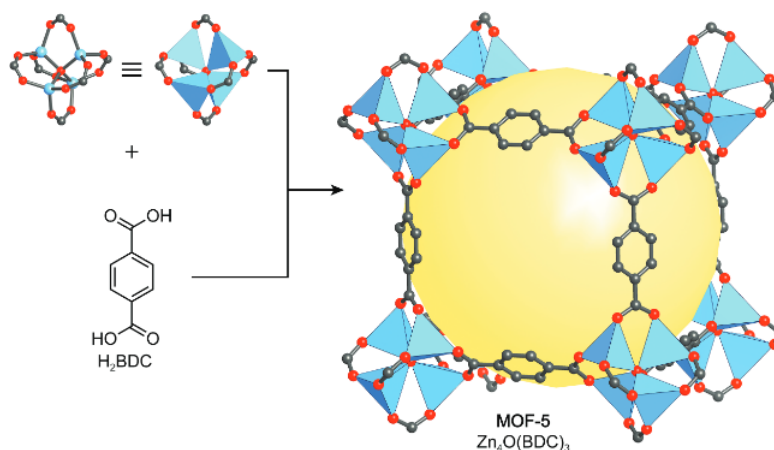


FIGURE 1.1: Unit cell structure of MOF-5 or IRMOF-1, one of the highest surface area to volume ratio among MOFs, at $2200 \text{ m}^2 \text{ cm}^3$ and the first MOF studied for hydrogen storage (Rosi et al. 2003).

2023) and their “reticular siblings” covalent organic frameworks (COFs)—composed only of light elements—show great promise as ANG solutions (Furukawa et al. 2009; Mendoza-Cortes et al. 2011; Martin et al. 2014; Tong et al. 2018).

1.2 The Problem

The intrinsic combinatorial character of reticular chemistry, translates to practically an infinite number of realizable structures. Currently, the Cambridge Structural Database (CSD) contains more than 100 000 experimentally synthesized MOFs (Moghadam, A. Li, et al. 2017) while the arrival of in silico designed MOFs (Wilmer et al. 2011; Colón et al. 2017; Boyd et al. 2019; Chung, Haldoupis, et al. 2019; Lee et al. 2021; Rosen et al. 2021; De Vos et al. 2023) has immensely expanded the available material pool. The huge size of current and future MOF databases (Lee et al. 2021) is both a blessing and a curse for the identification of novel materials. Blessing, since a large number of candidate structures doesn’t limit our choices and as such, the chances to find the right material for a given problem. Curse, since the enormous size of MOFs space makes it harder for researchers to efficiently explore it, complicating the tracing of materials with the desired properties. It is therefore crucial to find a way that allows us to efficiently explore such a huge material space (see FIGURE 1.2). Another way to rephrase our problem is the following: ***Given a large catalog of MOFs, is there a way to efficiently filter out the most promising ones for the application of interest?***

As a first approach to deal with this challenge, one could, in principle experimentally synthesize and characterize each one of the materials listed in the given catalog. Although *experimental synthesis and characterization* is the ultimate way to assess the performance of a material¹, the fact that a single laboratory study can take days or even months, renders experimental techniques impractical. A more efficient approach is computational screening based on *molecular simulations*, which for years has served

¹As Richard Feynmann said: “The test of all knowledge is experiment. Experiment is the sole judge of scientific truth”.

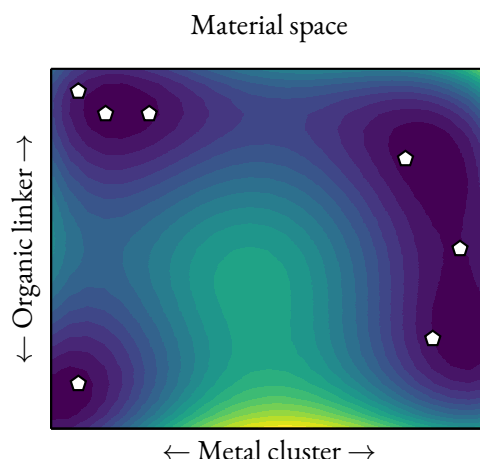


FIGURE 1.2: Material space of MOFs. Each point in this space corresponds to a unique combination of organic linker and metal cluster, whereas the associated color denotes the “score” of material (point) for a given application. Finding the best material (the pentagons) for a given application, amounts to solving a (probably) non-convex optimization problem.

as the principal tool for the discovery of high-performing MOFs (Simon, J. Kim, et al. 2015; Banerjee et al. 2016; Gómez-Gualdrón et al. 2016; Jeong et al. 2017; Moghadam, Islamoglu, et al. 2018). Although computational screening dramatically accelerates the assessment of a single material compared to experimental techniques, brute-force screening of current and upcoming databases is considered suboptimal, given the size of the latter.

Machine learning (ML) aka *data-driven techniques* come to the rescue when dealing with *big data* and over the last years have picked up the torch from molecular simulations regarding the screening of large databases. Given a collection of *input-output* pairs, i.e. a mathematical representation of a material and a corresponding property, a ML algorithm² seeks to *uncover the underlying structure-property relationship*. To put it in a nutshell, a ML algorithm “eats” *data*—which may come either from experiments, simulations or a combination of the two—and “spits out” a *model*, which can be *used to sort a large catalog of MOFs in just few seconds*. Obviously, for ML approaches to be effective and reliable, it is necessary that the resulting models are of high quality.

1.3 Literature Review

One of, if not the most important factor for the performance of ML models, is the way we select to mathematically represent materials or molecules. In other words, the type and amount of chemical information that is “injected” into these representations commonly known as *descriptors*, can make the difference between a high-performing and a baseline model. As such, it is of uttermost importance to employ descriptors that provide sufficient information for the properties of materials or molecules we are interested in to predict.

²Note that ML algorithms are not limited to solve only such kind of problems, which fall under the umbrella of supervised learning. See SECTION 2.1.1 for other types of problems tackled by ML.

With regards to gas adsorption in MOFs, one of the first and most commonly used descriptors, are the so called *geometric* ones, which aim to capture the pore environment of the framework. This type of descriptors includes textual characteristics of MOFs such as void fraction, gravimetric surface area and pore limiting diameter. Although ML models build with these descriptors work particularly well at the high pressure regime (Fernandez et al. 2013; Dureckova et al. 2019; Wu et al. 2020), their performance deteriorates when adsorption takes place at low pressures or the guest molecules exhibit non-negligible electrostatic interactions with the framework atoms. This performance drop should be expected, since geometric descriptors completely ignore the “cornerstone” of adsorption: *host-guest interactions*.

In order to improve the performance of ML models and bypass the limitations of geometric descriptors in the aforementioned conditions, another type of descriptors known as *energy-based* descriptors (Simon, Mercado, et al. 2015; Fanourgakis, Gkagkas, Tylianakis, Klontzas, et al. 2019; Orhan et al. 2023; Shi et al. 2023), has been introduced. This type of descriptors supply ML algorithms with information regarding the energetics of adsorption, and can be used standalone or in combination with geometric descriptors.

In one of the first works to fingerprint the energetic landscape of MOFs (Bucior, Bobbitt, et al. 2019), energy histograms derived from the interactions of guest molecules with the framework atoms were used to predict hydrogen and methane uptake with remarkable accuracy. Prior to calculating the energy histograms, a 3D grid is overlayed on the unit cell of the MOF. Next, at each point of the grid, the interaction between the guest molecule with the framework atoms is calculated, producing a 3D energy grid. *The latter is finally converted into a histogram, by partitioning the energy values of the grid into bins of specific energy width.* By using solely these histograms as descriptors—without including any textual property—Bucior, Bobbitt, et al. (2019) trained Lasso regression models, for predicting: i). H₂ swing capacity between 100 bar and 2 bar at 77 K ii). CH₄ swing capacity between 65 bar and 5.8 bar at 298 K. The resulting models were extremely accurate, achieving a mean absolute error (MAE) of 2.3 g L⁻¹ and 12.9 cm³ cm⁻³ for H₂ and CH₄, respectively, tested on the hMOFs database (Wilmer et al. 2011).

In another work (Fanourgakis, Gkagkas, Tylianakis, and Froudakis 2020), a set of descriptors based on the average interaction between fictitious probe particles and the framework atoms was introduced. Two different types of probe particles were proposed: i). Vprobe particles, which account for the van der Waals interactions ii). Dprobe particles, which are neutrally charged electric dipoles and account for the electrostatic interactions. Each of these fictitious probe particles is randomly inserted at different positions of the unit cell, and the interaction energy between the probe and the framework atoms is calculated. *The interaction energies at the different positions are averaged out, producing an energetic fingerprint of the material.* These fingerprints in combination with six geometric descriptors formed the input for the Random Forest (RF) algorithm, which was trained to predict gas uptake for a plethora of guest molecules and thermodynamic conditions, on the Computation-Ready Experimental (CoRE) MOF database (Chung, Camp, et al. 2014). The ML models achieved impressive performance, showing an R^2 value (see SECTION 3.3 for a definition) of: i). 0.874 for H₂ uptake at 77 K and 2 bar ii). 0.889 for CH₄ uptake at 298 K and 5.8 bar. A highlight of this work was the exceptional performance of the ML model with regards to CO₂ uptake at 300 K and 0.1 bar, achieving an R^2 score of 0.832. At the same conditions, the ML model trained with geometric descriptors only achieved an R^2 score of 0.507. That is, the “injection” of energetic information resulted in 60 % increase in accuracy.

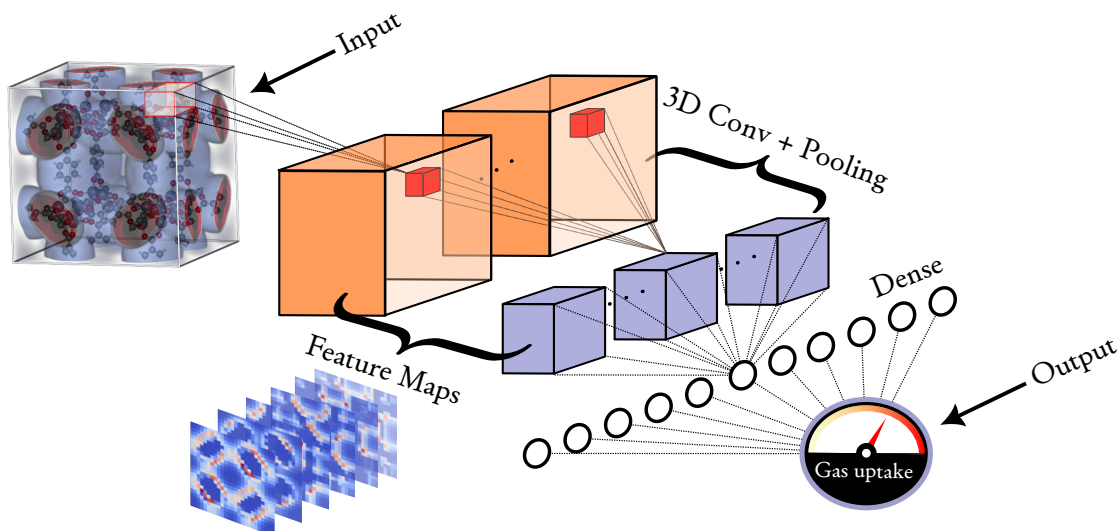


FIGURE 1.3: Proposed scheme to predict gas adsorption properties, starting from the PES as raw input. A 3D convolutional neural network (CNN) extracts its features from the PES, and then uses them to predict the adsorption property of interest. The IRMOF-1 structure and PES were visualized with the iRASP software (Dubbeldam et al. 2018).

1.4 Thesis Statement

In the aforescribed works, a general pattern can be recognized with regards to the building of the ML models. Starting from the potential energy surface (PES) or an approximation thereof, energetic fingerprints are manually handcrafted based on some heuristic, and these fingerprints are then used to train a ML algorithm. However, a lot of information has been lost during the conversion of the PES into fingerprints, as a 3D object is converted into an 1D object. *Since gas adsorption comes down to the PES, it is reasonable to question whether one can use the PES itself as descriptor.* By doing this: i). The information content that goes into a ML algorithm is increased ii). The computational cost remains the same relative to the previously described works iii). It is no longer necessary to manually handcraft fingerprints.

In this thesis, a generalized framework to predict gas adsorption properties is proposed, using the PES as raw input. Since the latter captures both the host-guest interactions and the geometry of the pore, i.e. the factors that mainly determine gas uptake in the low and high pressure regime (Broom et al. 2019), respectively, it can be regarded as the perfect input. In order to be machine understandable the PES is first voxelized—the voxelized PES is essentially a 3D energy image where each 3D pixel aka voxel, is colorized by energy—and then, it is processed by a 3D convolutional neural network, known for its ability to process image-like data. The proposed scheme is schematically presented in FIGURE 1.3.

Chapter 2

Theoretical Background



DEEP LEARNING, a class of machine learning algorithms based on neural networks, has revolutionized the way we tackle a problem from a ML perspective and is one if not the most important factor for recent ML achievements. Solving complex tasks such as image classification or language translation, that for years have bedevilled traditional ML algorithms, constitutes the signature of deep learning (DL). Admittedly, *the advent of a deep convolutional neural network*, the AlexNet (Krizhevsky et al. 2012) on September 30 of 2012, signified the “modern birthday” of this field. On this day, AlexNet not only won the ImageNet (Deng et al. 2009) Large Scale Visual Recognition Challenge (ILSVRC), but dominated it, achieving a top-5 accuracy of 85 %, surpassing the runner-up which achieved a top-5 accuracy of 75 %. AlexNet showed that neural networks (NNs) are not merely a pipe-dream, but they can be applied in real-world problems. It is worth to notice that ideas of NNs trace back to 1943, but it was until recently that these ideas got materialized. The reason for this recent breakthrough of DL (and ML) is twofold. First, the availability of large datasets—the era of big data—such as ImageNet. Second, the increase in computational power, mainly of GPUs for DL, accelerating the training of deep NNs and traditional ML algorithms.

2.1 Machine Learning Preliminaries

Since DL is a subfield of ML, it is necessary to familiarize with the later before diving into the former. In this section, the necessary theoretical background and jargon of ML is presented. Machine learning can be defined as “*the science and (art) of programming computers so they can learn from the data*” (Géron 2017). A more technical definition is the following:

DEFINITION 2.1 (Machine learning, Mitchell 1997). *A computer program is said to learn from experience E with respect to some class of tasks T and some performance measure P , if its performance on T , as measured by P , improves with experience E .*

For instance, a computer program that classifies emails into spam and non-spam (the task T), can improve its accuracy, i.e. the percentage of correctly classified emails (the performance P), through examples of spam and non-spam emails (the experience E). But in order to take advantage of the experience aka *data*, it must be written in such a way that *adapts to the patterns in the data*. Certainly, a *traditional spam filter can not learn from experience*, since the latter does not affect the classification rules of the former and as such, its performance. For a traditional spam filter to adapt to new patterns and perform better, it must change its hard-wired rules, but by then it will be a different program. In contrast, a *ML-based filter can adapt to new patterns, simply because it has been programmed to do so*.

In other words, *in traditional programming we write rules for solving T whereas in ML we write rules to learn the rules for solving T* . This subtle but essential difference is what gives ML algorithms the ability to take advantage of the data.

2.1.1 Learning paradigms

Depending on the type of experience they are allowed to have during their *training phase* (Goodfellow et al. 2016), ML approaches are divided into three main *learning paradigms*: **unsupervised learning**, **supervised learning** and **reinforcement learning**. The following definitions are not by any means formal, but merely serve as an intuitive description of the different paradigms.

DEFINITION 2.2 (Unsupervised learning). *The experience comes in the form $\mathcal{D}_{\text{train}} = \{\mathbf{x}_i\}$, where $\mathbf{x}_i \sim p(\mathbf{x})$ is the input of the i -th training instance aka sample. In this paradigm we are interested in learning useful properties of the underlying structure captured by $p(\mathbf{x})$ or $p(\mathbf{x})$ itself.*

For example, suppose we are interested in generating images that look like Picasso paintings. In this case, the input is just the pixel values, i.e. $\mathbf{x} \in \mathbb{R}^{W \times H \times 3}$. The latter follow a distribution $p(\mathbf{x})$, so all we have to do is to train an unsupervised learning algorithm with many Picasso paintings to get a *model*, that is $\hat{p}(\mathbf{x})$. Assuming the estimation of the original distribution is good enough, new realistically looking paintings (with respect to original Picasso paintings) can be “drawn” by just sampling from $\hat{p}(\mathbf{x})$. In the ML parlance, this task is known as *generative modeling* while inputs are also called *features*, *predictors* or *descriptors*.

DEFINITION 2.3 (Supervised learning). *The experience comes in the form $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$, where $(\mathbf{x}_i, \mathbf{y}_i) \sim p(\mathbf{x}, \mathbf{y})$ and $\mathbf{y}_i$ is the output aka label of the i -th training instance. In this paradigm we are usually interested in learning a function $f: \mathcal{X} \rightarrow \mathcal{Y}$.*

This paradigm comes mainly under two flavors: *regression* and *classification*, which are schematically depicted in FIGURE 2.1. In regression the interest is in predicting a continuous value given an input, i.e. $y \in \mathbb{R}$, such as a molecular property given a mathematical representation of a molecule. In classification, the interest is to predict in which of k classes an input belongs to, i.e. $y \in \{1, \dots, k\}$, such as predicting the breed of a dog image given the raw pixel values of the image. The term “supervised” is coined due to the “human supervision” the algorithm receives during its training phase, through the presence of the correct answer (the label) in the experience. In a sense, in this paradigm we “teach” the learning algorithm aka *learner*. It should be emphasized that the label is not constrained to be single-valued, but can also be multi-valued. In this case, one talks about *multi-label regression* or *classification* (Read et al. 2009).

A more exotic form of supervised learning is *conditional generative modelling*, where the interest is in estimating $p(\mathbf{x} | \mathbf{y})$. For example, one may want to build a model that generates images of a specific category or a *model that designs molecules/materials with tailored properties* (K. Kim et al. 2018; Yao et al. 2021; Gebauer et al. 2022). This is one approach of how ML can tackle the *inverse design problem* in chemistry.

DEFINITION 2.4 (Reinforcement learning). *The experience comes from the interaction of the learner, called agent in this context, with its environment. In other words, there is a feedback loop between the learner and its environment. In this paradigm we are interested in building an agent that can take suitable actions in a given situation.*

The agent observes its *environment*, selects and performs *actions* and gets *rewards* or *penalties* in

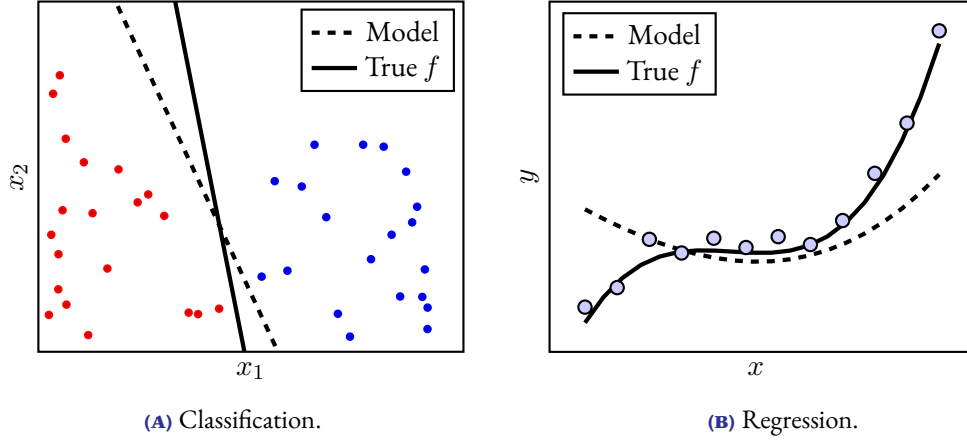


FIGURE 2.1: Main tasks of supervised learning.

return. The goal is to learn an optimal strategy, called a *policy*, that *maximizes the long-term reward* (Géron 2017). A policy simply defines the action that the agent should choose in a given situation. In contrast to supervised learning, where the correct answers are provided to the learner, *in reinforcement learning the learner must find the optimal answers by trial and error* (Bishop 2007). Reinforcement learning techniques find application in fields such as gaming (AlphaGo is a well known example), robotics, autonomous driving and recently chemistry (H. Li et al. 2018; Gow et al. 2022). Since in the present thesis only supervised learning techniques were employed, the remaining of this chapter is devoted to this learning paradigm.

2.1.2 Formulating the problem of supervised learning

The general setting of supervised learning is as follows: we assume that there is some relationship between $\mathbf{x}$ and $\mathbf{y}$:

$$\mathbf{y} = f(\mathbf{x}) + \epsilon \quad (2.1)$$

and we want to estimate f from the data. The function f represents the *systematic information* that $\mathbf{x}$ gives about $\mathbf{y}$ while ϵ is a *random error term* independent of $\mathbf{x}$ and with zero mean. More formally, we have an input space $\mathcal{X}$, an output space $\mathcal{Y}$ and we are interested in learning a function $\hat{h}: \mathcal{X} \rightarrow \mathcal{Y}$, called the *hypothesis*, which produces an output $\mathbf{y} \in \mathcal{Y}$ given an input $\mathbf{x} \in \mathcal{X}$. At our disposal we have a collection of input-output pairs $(\mathbf{x}_i, \mathbf{y}_i)$, forming the **training set** denoted as $\mathcal{D}_{\text{train}}$, with the pairs drawn i.i.d from $p(\mathbf{x}, \mathbf{y})$.

Ideally, we would like to learn a hypothesis that minimizes the **generalization error or loss**:

$$\mathcal{L} := \int_{\mathcal{X} \times \mathcal{Y}} \ell(h(\mathbf{x}), \mathbf{y}) p(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \quad (2.2)$$

that is, the expected value of some *loss function* ℓ over all possible input-output pairs. A loss function just measures the discrepancy of the prediction $h(\mathbf{x}) = \hat{\mathbf{y}}$ from the true value $\mathbf{y}$ and as such, the best hypothesis is the one that minimizes this integral. Obviously, it is impossible to evaluate the integral in EQUATION 2.2, since in general we don't know the joint probability distribution $p(\mathbf{x}, \mathbf{y})$.

The idea is to use the *training error or loss*:

$$\mathcal{L}_{\text{train}} := \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{i \in \mathcal{D}_{\text{train}}} \ell(h(\mathbf{x}_i), \mathbf{y}_i) \quad (2.3)$$

as an approximation for the generalization loss, and *choose the hypothesis that minimizes the training loss*, a principle known as *empirical risk minimization*. In other words, to get a hypothesis aka *model* $\mathcal{M}$ from the data, we need to solve the following optimization problem:

$$\hat{h} \leftarrow \arg \min_{h \in \mathcal{H}} \mathcal{L}_{\text{train}} \quad (2.4)$$

which is achieved by just feeding the training data into the learning algorithm $\mathcal{A}$:

$$\mathcal{M} \leftarrow \mathcal{A}(\mathcal{D}_{\text{train}}) \quad (2.5)$$

2.1.3 Components of a learning algorithm

By breaking down EQUATION 2.4, i.e. the optimization problem the learner needs to solve, the components of a learner can be revealed. The latter is comprised of the following three “orthogonal” components: a ***hypothesis space***, a ***loss function*** and an ***optimizer***. We now look into each of them individually and describe the contribution of each one to the solution of the optimization problem. For the ease of notation and clarity, in the remaining of this chapter we will stick to examples from simple (single-valued) regression and binary classification.

DEFINITION 2.5 (Hypothesis space). *The set of hypotheses (functions), denoted as $\mathcal{H}$, from which the learner is allowed to pick the solution of EQUATION 2.4.*

A simple example of a hypothesis space, is the one used in univariate *linear regression*:

$$\hat{y} = b + wx \quad (2.6)$$

where $\mathcal{H}$ contains all lines (or hyperplanes in the multivariate case) defined by EQUATION 2.6. Of course, one can get a *more expressive* hypothesis space, by including polynomial terms, e.g.:

$$\hat{y} = b + w_1x + w_2x^2 \quad (2.7)$$

The more expressive the hypothesis space, the larger the ***representational capacity*** of the learning algorithm. For a formal definition of representational capacity, the interested reader can look at *Vapnik-Chervonenkis Dimension* (Hastie et al. 2009).

DEFINITION 2.6 (Loss function). *A function that maps a prediction into a real number, which intuitively represents the quality of a candidate hypothesis.*

For example, a typical loss function used in regression is the *squared loss*:

$$\ell(\hat{y}, y) := (\hat{y} - y)^2 \quad (2.8)$$

where $y, \hat{y} \in \mathbb{R}$. A typical loss function for binary classification is the *binary cross entropy loss*:

$$\ell(\hat{y}, y) := -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})] \quad (2.9)$$



where $y \in \{0, 1\}$, indicating the correct class, and $\hat{y} \in [0, 1]$ which corresponds to the predicted probability for class-1. Notice that in both cases the loss is minimum when the prediction is equal to the ground truth. For the cross entropy loss, if $y = 1$ is the correct class, then the model must predict $\hat{y} = 1$ for the loss to be minimized.

Usually, we are not only penalizing a hypothesis for its mispredictions, but also for its *complexity*. This is done in purpose, since a learner with a *very rich hypothesis space* can easily memorize the training set but fail to generalize well to new unseen examples. *Every modification that is made to a learner in order to reduce its generalization loss but not its training loss, is called **regularization*** (Goodfellow et al. 2016).

A common—but not the only—way to achieve that, is by including another penalty term called *regularization term* or **regularizer**, denoted as $\mathcal{R}$, in EQUATION 2.3:

$$\mathcal{L}_{\text{train}} := \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{i \in \mathcal{D}_{\text{train}}} \ell(\hat{y}_i, y_i) + \lambda \mathcal{R} \quad (2.10)$$

The λ factor controls the strength of regularization and it is an **hyperparameter**, i.e. a parameter that is not learned during training but *whose value is used to control the training phase*. In order to see how λ penalizes model complexity, assume we perform univariate polynomial regression of degree k :

$$\hat{y} = b + \sum_{i=1}^k w_i x^i \quad (2.11)$$

combining mean squared loss (MSL) and *lasso regularization* as training loss:

$$\mathcal{L}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{i \in \mathcal{D}_{\text{train}}} \ell(\hat{y}_i - y_i)^2 + \lambda \sum_{i=1}^k |w_i| \quad (2.12)$$

Let's apply a very strong regularization by setting $\lambda \rightarrow \infty$ (in practice we set λ to a very large value) and observe what happens to the *weights* w_i . By setting $\lambda \rightarrow \infty$, the regularization term dominates the MSL and as such, the only way to minimize the training loss is by setting $w_i = 0$. This leave us with a very simple model—only the *bias* term b survives—which always predicts the mean value of y in the training set.

Applying a regularizer, is also useful when we need to select between two (or more) competing hypotheses that are equally good. For example, assuming two hypotheses achieve the same (unregularized) training loss, the inclusion of a regularization term help us decide between the two, *by favoring the simplest one*. This is reminiscent of the **Occam's razor** aka **principle of parsimony**, which advocates that between two competing theories with equal explanatory power, one should prefer the one with the fewest assumptions.

DEFINITION 2.7 (Optimizer). *An algorithm that searches through $\mathcal{H}$ for the solution of EQUATION 2.4.*

Having defined the set of candidate models (the hypothesis space) and a measure that quantifies the quality of a given model (the loss function), all that is remaining is a tool to scan the hypothesis space and pick the model that minimizes the training loss (the optimizer). A naive approach is to check all hypotheses in $\mathcal{H}$ and then pick the one that achieves the lowest training loss. This approach can

work if $\mathcal{H}$ is finite, but obviously doesn't scale in the general case where $\mathcal{H}$ is infinite¹. More efficient approaches are needed if we are aiming to solve EQUATION 2.4 in finite time.

One optimizer that is frequently used in ML and is the precursor of more refined ones, is **gradient descent**. With this method, the exploration of hypothesis space² involves the following steps:

ALGORITHM 1: Gradient descent

```

1  $\theta \leftarrow$  random initialization;
2 while stopping criterion not met do
3    $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{train}}(\theta)$ ;
4 end
```

where η is a small number called the *learning rate*. Gradient descent is based on the idea that if a multivariate function is defined and differentiable at a point $\mathbf{x}$, then $f(\mathbf{x})$ decreases fastest if one takes a small step from $\mathbf{x}$ in the direction of negative gradient at $\mathbf{x}$, $-\nabla f(\mathbf{x})$.

The motivation becomes clear if we look at the differential of $f(\mathbf{x})$ in direction $\mathbf{u}$:

$$f(\mathbf{x} + \delta \mathbf{u}) - f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \delta \mathbf{u} \quad (2.13)$$

EQUATION 2.13 says that this differential is minimized³ when $\delta \mathbf{u}$ is anti-parallel to $\nabla f(\mathbf{x})$ and that is why we subtract the gradient in ALGORITHM 1, i.e. move in direction anti-parallel to the gradient. The fact that EQUATION 2.13 holds only locally (magnitude of $\delta \mathbf{u}$ must be small) explains why η must be a small number. It should be added that gradient descent can be trapped to (potential) local minima of the training loss and therefore fail to solve EQUATION 2.4. As it will be discussed later, this is not a problem, because *the ultimate purpose is to find a hypothesis that generalizes well, not necessarily the one that minimizes the training loss*⁴. Optimizers are discussed in further detail in SECTION 2.2.4.

Before moving on, it is worth to add that both the regularization and the optimizer have an effect on the “true” or **effective capacity** (Goodfellow et al. 2016) of the learner, *which might be less than the representational capacity of the hypothesis space*. For example a regularizer penalizes the complexity of an hypothesis, effectively “shrinking” the representational capacity of the hypothesis space. The effect of the optimizer can be understood by looking on its contribution to the solution of EQUATION 2.4. As described previously, the optimizer searches through the hypothesis space. If this “journey” is not long enough, then this “journey” is practically equivalent to a long “journey” in a shortened version of the original hypothesis space. In the rest of this chapter, by the term **complexity** or *capacity of a learner*, we mean its effective capacity, which is affected by all its three components.

2.1.4 Performance, complexity and experience

Suppose that we have trained our learner, and finally we get our model, as stated by EQUATION 2.5. *How can we assess its performance?* Remember, we can't calculate the generalization loss, since we are not given an infinite amount of data. First of all, *we should not report the training loss, because it is*

¹It is not uncommon for $\mathcal{H}$ to be infinite. Even for simple learners like linear regression $\mathcal{H}$ is infinite, since there infinite lines defined by EQUATION 2.6.

²We have implicitly assumed that $\mathcal{H}$ can be parameterized, i.e. $\mathcal{H} = \{h(\mathbf{x}; \theta) \mid \theta \in \Theta\}$, where Θ denotes the parameter space, the set of all values the parameter θ can take. This allows us to write the training loss as function of model parameters and optimize them with gradient descent.

³The right hand side of EQUATION 2.13 is a dot product.

⁴Remember we use the training loss (see EQUATION 2.3) as a proxy for the generalization loss (see EQUATION 2.2).

optimistically biased, as it is evaluated on the same data that has been trained on⁵. What we should do is to collect new input-output pairs, forming the **test set** $\mathcal{D}_{\text{test}}$, and then *estimate the generalization loss* as following:

$$\mathcal{L}_{\text{test}} := \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i \in \mathcal{D}_{\text{test}}} \ell(h(\mathbf{x}_i), \mathbf{y}_i) \quad (2.14)$$

The *test error or loss* is evaluated on new—unseen to the learner during the training phase—samples and as such, it is an *unbiased estimate* of the generalization loss. Usually, since many times is not even possible to collect new samples, *we split the initial dataset into training and test sets*.

The general recipe for building and evaluating the performance of a ML model has already been presented. What is missing is how we can improve its performance, or to put it differently, the factors that affect the quality of the returned model. There are two main factors that determine the performance of the model: *complexity and experience*. In general, the larger the experience—the training set—the better the performance, just like we humans perform improve on a task by keep practicing. With regards to the complexity, *learners of low complexity might fail to capture the patterns in the data*, meaning that the resulting model will fail to generalize. In contrast, *learners of higher complexity would be able to capture these patterns*, and as such return models of higher quality. However, *as the complexity of the learner keeps increasing, the latter is more sensitive to noise, i.e. there is a higher chance that the learner will simply memorize its experience* and as such, fail to generalize.

In other words, there is a trade-off between the complexity of the learner and its performance. The learner should be not too simple but also not too complex, in order to generalize well. This in turn implies that we need to find a way to “tune” the complexity. A common way to achieve that is by using another set of instances, known as the **validation set**. We train learners of different complexity on the training set, evaluate their performance on the validation set, and then choose the learner that performs best on the validation set. The reason we use the validation set instead of the test set for tuning complexity, is to ensure that the performance estimation is unbiased. *The test set should not influence our decisions in any way*. After we have tuned the complexity, we can estimate the performance of the resulting model in the test set. Finally, it is a good practice to retrain the learner on the whole dataset—including validation and test sets—since more data result in models of higher quality.

THEOREM 1 (Bias-variance decomposition, Bishop 2006). *From EQUATION 2.1 the expected squared loss at $\mathbf{x}^*$ can be decomposed as following:*

$$\mathbb{E} \left[\left(y^* - \hat{f}(\mathbf{x}^*) \right)^2 \right] = \left(f(\mathbf{x}^*) - \mathbb{E} \left[\hat{f}(\mathbf{x}^*) \right] \right)^2 + \mathbb{E} \left[\left(\hat{f}(\mathbf{x}^*) - \mathbb{E} \left[\hat{f}(\mathbf{x}^*) \right] \right)^2 \right] + \sigma_\epsilon^2 \quad (2.15)$$

The expected squared loss refers to the average squared loss we would obtain by repeatedly estimating f using different training sets, each tested at $\mathbf{x}^$. The overall expected squared loss can be computed by averaging the left hand side of EQUATION 2.15 over all possible values $\mathbf{x}^*$.*

The trade-off between the complexity of the learner and its performance, is mathematically described in THEOREM 1. EQUATION 2.15 states that the error of the learner can be decomposed into three terms: **bias**, **variance** and **irreducible error**. The bias (squared)—first term of EQUATION

⁵Intuitively, this is like assessing students’ performance based on problems they have already seen before. They can easily achieve zero error, just by recalling their memory.

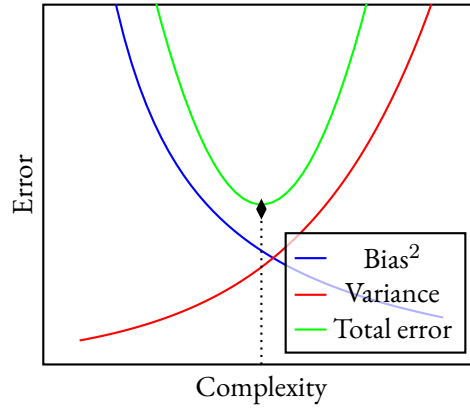


FIGURE 2.2: The bias-variance trade-off. For a given task, there is a “sweetspot” of complexity that minimizes the total error. Bias^2 and variance correspond to the first and second term of EQUATION 2.15, respectively.

2.15—refers to the error introduced by *approximating a real-world problem, which can be highly complicated, by a much simpler model* (Hastie et al. 2009; James et al. 2014). For instance, if the input-output relationship is highly nonlinearity, using linear regression to approximate f , will undoubtedly introduce some bias in the estimate of f . In contrast, if the input-output relationship is very close to linear, linear regression should be able to produce an accurate estimate of f . In general, more flexible learners, result in less bias (Hastie et al. 2009; James et al. 2014).

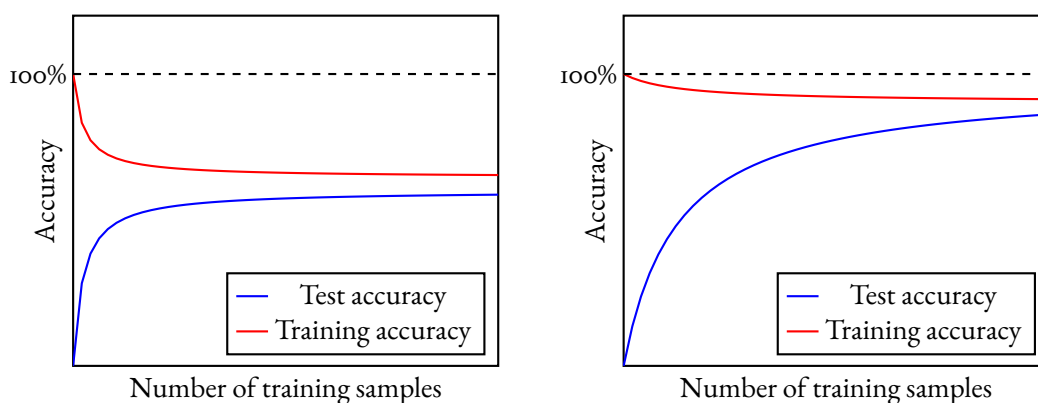
The variance—second term of EQUATION 2.15—refers to the *degree by which $\hat{f}$ would change if it was estimated by different training sets*. Since the training data are used to fit the learning algorithm, different training sets will result in a different estimate of f . Ideally, $\hat{f}$ should not exhibit too much variation between different training sets, since otherwise small changes in the training can result in large changes in $\hat{f}$. In that case, the learner essentially memorizes the training data. Generally, more flexible learners, result in higher variance (Hastie et al. 2009; James et al. 2014).

Lastly, the irreducible error—third term of EQUATION 2.15—refers to the *error caused by stochastic label noise*, as can be seen from EQUATION 2.1. A possible source for this noise, might be omitted features which are useful in predicting the output. It is called irreducible, because no matter how well we estimate f , even if we predict $\hat{y} = f(\mathbf{x})$, we can’t reduce the error associated to the variability of ϵ . As stated in SECTION 2.1.2, this random error term is independent of $\mathbf{x}$, and as such, we have no control over it. The *bias-variance trade-off* is schematically depicted in FIGURE 2.2. Interested readers might also appreciate reading about *double descent* (Nakkiran et al. 2019), a phenomenon where increasing further the complexity of the learner, results in a new minimum (hence, the name).

FIGURE 2.3, shows the *learning curves* for learners of different complexity. A learning curve is a plot of the training and test performance⁶ of the learner as function of its experience. First, let’s look at the learning curve of the low complexity learner. The accuracy⁷ starts out high on the training set, since with a small number of samples, the learner can fit them perfectly. However, by adding more training data, learner’s training accuracy quickly drops due to learners inflexibility and inexpressivity.

⁶Usually, only the test performance is plotted.

⁷By accuracy we mean any performance metric where higher values are better, not necessarily the classification accuracy.



(A) Learning curve of learner with low complexity. (B) Learning curve of learner with high complexity.

FIGURE 2.3: Relation between performance and experience.

That is, it can't fit the patterns in the training data. On the other hand, test accuracy starts out very low since with very few training data, it is unlikely that the training set is a good representation of the underlying distribution $p(\mathbf{x}, \mathbf{y})$. In other words, it is unlikely that the learner will experience patterns in the training data, that will help it to generalize well. By increasing the training data, test accuracy increases but it never reaches a high value. This happens due to the learner's inability to detect and exploit the patterns in the training data. In other words, the learner fails to generalize well not because its experience is low, but because it is biased. That is, it oversimplifies the problem and makes strong assumptions that do not capture the complexity of the data.

Now let's consider the learning curve of the high complexity learner. Again, the training accuracy starts out high with a small amount of training data. However, in contrast to the previous case, as the number of training samples increases, the training accuracy remains high since the learner is flexible enough to learn the patterns in the training set, irrespective of its size. At some point, the training data becomes large enough that it is a good representation of the underlying distribution $p(\mathbf{x}, \mathbf{y})$ and since the learner is very flexible, it can capture the true patterns in $p(\mathbf{x}, \mathbf{y})$, increasing the test accuracy. It is worth pointing out that the learning and complexity curves (see FIGURE 2.2), are just two slices of the same 3D plot: the plot of performance as function of experience and complexity.

2.2 Fundamentals of Deep Learning

Having covered the basic jargon and concepts of ML, we are now in a position to dive into DL. One might expect that DL is a very complex subfield of ML, given its astonishing results in complex tasks, but quite the opposite holds. Notably, DL shares similar ideas with reticular chemistry: *combining simple computational units, known as **neurons**, to achieve intelligent behavior*. And just like we can tune the properties of MOFs by judiciously selecting and combining their building blocks, we can design problem-specific *neural architectures* by reasonably arranging and connecting the neurons. In other words, both DL and reticular chemistry can be viewed as building with Legos.

Since the term "neuron" is admittedly a neuroscience term, one might wonder what is the relation between DL and the human brain. The neural perspective on DL is mainly motivated by the

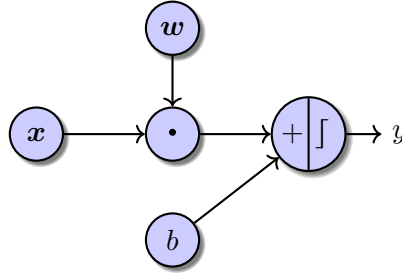


FIGURE 2.4: The perceptron.

following idea: *the brain is a proof by example that intelligent behavior is possible and as such, a straightforward approach to build an intelligent system is by reverse engineering the computational principles behind the brain and duplicating its functionality*. However, the term “deep learning” is not limited to this neuroscientific perspective. It appeals to a more general principle of learning *multiple levels of abstraction*, which is applicable in ML frameworks that are not necessary neurally inspired (Goodfellow et al. 2016).

DEFINITION 2.8 (Deep learning). *Class of machine learning algorithms inspired by brain organization, based on learning multiple levels of representation and abstraction. They achieve great power by learning to represent the world⁸ as a nested hierarchy of concepts.*

Before exploring NNs we first need to understand how the neuron, the basic building block of NNs, works. *A neuron is nothing more than a device—a simple computational unit—that makes decisions by weighing up evidence* (Nielsen 2018). This sounds very similar to the way humans make decisions. For instance, suppose the weekend is coming up and your favorite singer has scheduled a concert near your city. In order to decide whether you should go to the concert or not, you weigh up different factors, such as weather conditions, ease of transportation (you don’t own a car) and whether your boyfriend or girlfriend is willing to accompany you. This kind of decision-making can be described mathematically as following:

$$\text{decision} = \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b > 0 \\ 0 & \text{otherwise} \end{cases} \quad \text{where } \mathbf{w}^\top \mathbf{x} := \sum_i w_i x_i \quad (2.16)$$

If this weighted sum plus the bias⁹—your willing to go to the festival irrespective of the evidence—is greater than zero, then your decision is positive, otherwise negative.

The simple decision-making rule specified by EQUATION 2.16, which is known as the **perceptron** (Rosenblatt 1957), is schematically depicted in FIGURE 2.4. Essentially, the perceptron is a linear binary classifier (see FIGURE 2.1a). If we pay a little more attention to EQUATION 2.16, we can see that that the decision is basically an *application of a linear function*¹⁰ followed by a *nonlinearity*. As such, we can rewrite EQUATION 2.16 as following:

$$y = \phi(\mathbf{w}^\top \mathbf{x} + b) \quad (2.17)$$

⁸Hierarchy is deeply rooted in our world. Just think the hierarchy from subatomic particles to macroscopic objects.

⁹If you prefer the neuroscientific analogy, you can think of bias as how easy is for a neuron to “fire”.

¹⁰Formally speaking it is an affine function. We can turn it into a linear by “absorbing” the bias term into the weights and adding 1 to the input vector, a procedure known as the *bias trick*.

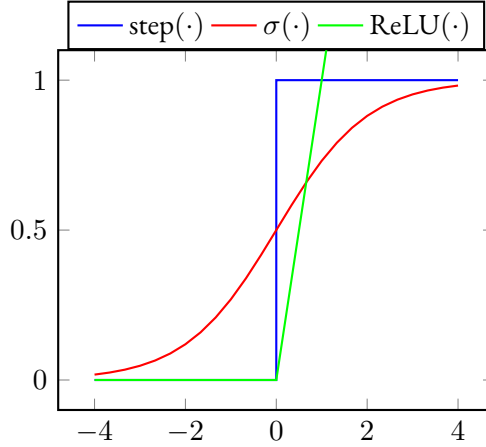


FIGURE 2.5: Examples of activation functions.

where $\phi(\cdot)$ is the nonlinear function aka **activation function**.

In the perceptron, the activation function is the Heavyside step function but in modern NNs it has been substituted by functions such as the sigmoid, hyperbolic tangent and currently the rectified linear unit (ReLU) and its variants. Some activation functions are graphically shown in FIGURE 2.5. The reason that the step function isn't used anymore is that its derivative vanishes everywhere, which is problematic for gradient-based optimization methods that power the training of modern NNs. The ReLU function is defined as:

$$\text{ReLU}(x) := \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \quad (2.18)$$

A common variant of ReLU is the leaky rectified linear unit (LeakyReLU) function which is defined as:

$$\text{LeakyReLU}(x) := \max(0, x) + a \min(0, x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{otherwise} \end{cases} \quad (2.19)$$

where a is a small positive constant usually set to 0.01. It is worth to notice how simple the nonlinearities used in NNs are. For instance, ReLU, the most commonly used activation function these days, is just a piecewise linear function. This again highlights the fundamental idea behind DL: *building something complex by combining simple elements*.

2.2.1 Neural networks

Neural networks can be thought as **collection of neurons** organized in layers and can be represented as *computational graphs*.

DEFINITION 2.9 (Graph). *A set of objects in which some pairs of objects are in some sense “related”. See FIGURE 2.7 for some types of graphs.*



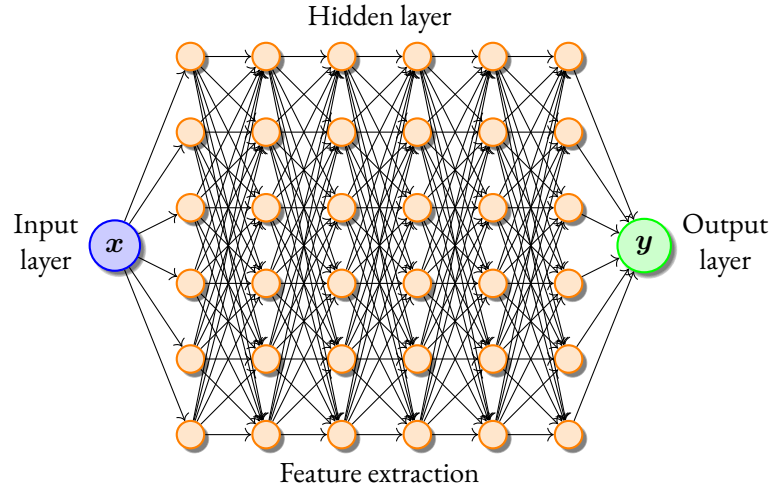


FIGURE 2.6: The multilayer perceptron. A typical example of a neural network.

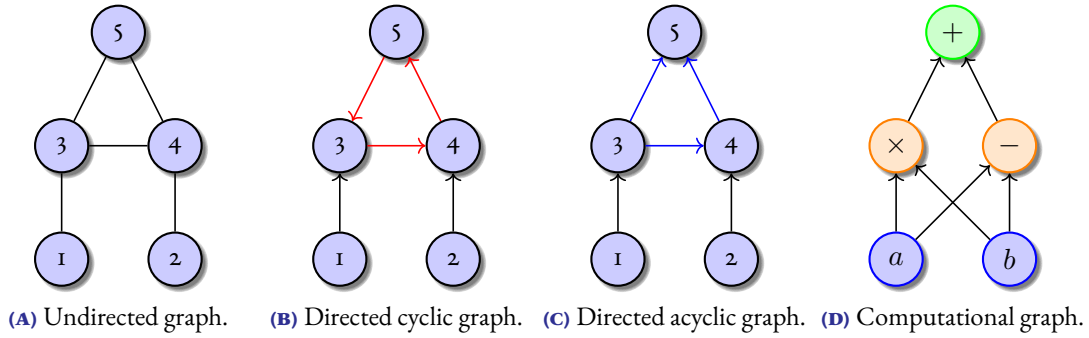


FIGURE 2.7: Examples of graphs. In a directed graph the edges have direction. If at least one loop is present, they are called cyclic, otherwise acyclic.

DEFINITION 2.10 (Computational graph). *A directed acyclic graph (DAG) where nodes correspond to operations or variables and edges show the data flow between the nodes.*

In general, the architecture of a NN can be broken down into the following three layers: **input layer**, **hidden layer** and **output layer**. Information flow starts from the input layer, passes through the hidden layer(s) and finally ends at the output layer. Neural networks with more than one hidden layer are classified as deep, and shallow otherwise. A typical architecture known as multilayer perceptron (MLP)¹¹ or fully connected neural network (FCNN) is presented in FIGURE 2.6. It should be emphasized that all the neurons in the hidden layers aka *hidden units* of the network perform exactly the same operation as that of the perceptron, described in EQUATION 2.17. As such all the *hidden units* at a given layer make decisions based on the decisions of the previous layer. Unsurprisingly, the kind of decisions made by the neurons depends solely on the problem and the data distribution at hand.

¹¹The term MLP is kind of a misnomer, since the step function used originally in the perceptron is no longer used in modern NNs.

To understand better the purpose of the hidden layers and the functionality of a NN as a whole, let's consider the problem of image classification. This is by no means a trivial task, since we need to learn a mapping from a set of pixels to an object identity. Imagine for a moment you are blindfolded and you need to classify an image. The valid classes are: “person”, “car” and “ship”. Furthermore, assume that the correct class is “person”. *Would you prefer to hear the sequence of pixel values or whether the image contains a face?* In other words, it is a lot easier to classify the content of an image if we know some *high-level features*, i.e. *a high-level description of the image*. *Neural networks extract such high-level features by exploiting the hierarchical structure of images*. A complex object like a “face” is defined in term of simpler ones, such as “eye” and “nose”, which in turn are defined in terms of simpler ones and so on. This hierarchy allows the NN to solve the complex task of image classification by breaking it down into smaller sub-problems. The first layer learns to detect edges. The second layer combines the decisions of the first layer to detect corners. Subsequently, the third layer combines the decisions of the second layer to identify shapes like circles and squares and so on, until we reach the final layer which is able to detect high-level features such as objects or object parts. The deeper we are into the network—i.e. the closer to the output layer—the more abstract and task-specific the detected features become.

In a FCNN with N hidden layers, each hidden layer performs the following operation:

$$\mathbf{h}^l = \phi \left(W^l \mathbf{h}^{l-1} + \mathbf{b}^l \right) \quad \text{where} \quad 1 \leq l \leq N \quad \text{and} \quad \mathbf{h}^0 := \mathbf{x} \quad (2.20)$$

which is just a matrix version of EQUATION 2.17 with the matrix W^l playing the role of the “synapses” between the neurons of the layers $l - 1$ and l . Since the “stacking” of many hidden layers is equivalent to a huge composite function:

$$\tau(\mathbf{x}) := \left(\mathbf{h}^l \circ \mathbf{h}^{l-1} \dots \circ \mathbf{h}^1 \right) (\mathbf{x}) \quad (2.21)$$

and the output layer is just a linear function of the last hidden layer, the output of the FCNN can be written as:

$$\hat{y} = \mathbf{w}^\top \tau(\mathbf{x}) + b \quad (2.22)$$

or in the general case of multi-valued output:

$$\hat{\mathbf{y}} = W \tau(\mathbf{x}) + \mathbf{b} \quad (2.23)$$

In other words, *a linear model on top of the extracted features*. It should be emphasized that EQUATION 2.23 is not specific to FCNNs, but describes every type of NN used for classification and regression. Moreover, the use of activation function now becomes more clear: *the composition of many linear functions is just another linear function, which implies nonlinearities must be inserted between them, if we aim to learn a nonlinear relationship*. EQUATION 2.23 can also be understood in the following way: *a problem that is nonlinear—i.e. complex—in the original space, can become linear—i.e. simple—in a transformed space*. FIGURE 2.8 shows such an example, known as the XOR problem. The solution of EQUATION 2.23 essentially boils down to finding the right transformation function $\tau(\mathbf{x})$. Please note that traditional ML algorithms like support vector machines (SVMs), also map the original space to a transformed space. However, they use a *fixed—i.e. not learnable during the training phase—mapping*. *Deep learning algorithms on the other hand learn this mapping during their training phase, considering both the problem and the data at hand*.

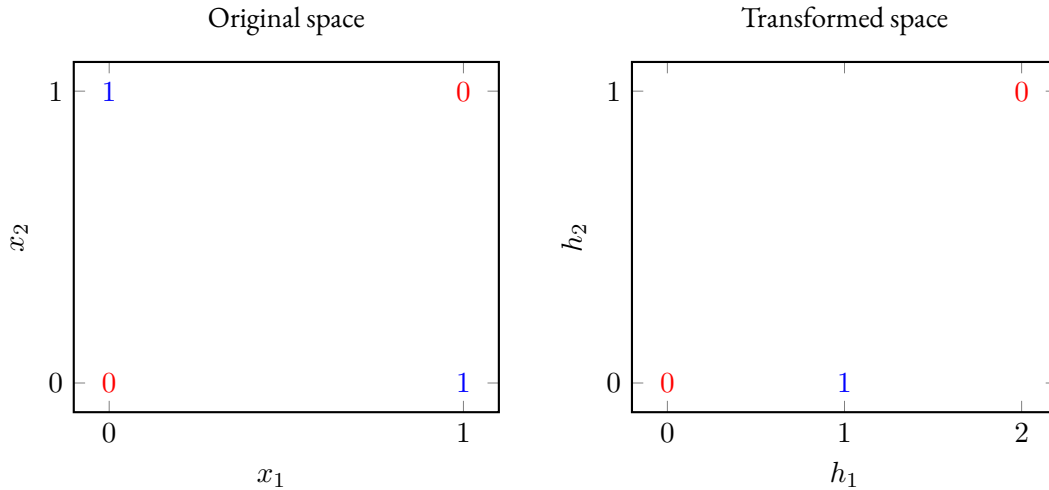


FIGURE 2.8: Solving the XOR problem. A linear classifier in the original space can’t perfectly separate the “ones” and “zeros”. In contrast, if the points are projected into a new space, then they become linearly separable.

THEOREM 2 (Universal approximation theorem, Hornik et al. 1989). *A feedforward¹² network with a single hidden layer containing a finite number of neurons can approximate any continuous function.*

One interesting fact about NN is summarized in THEOREM 2. An informal proof goes like this. The value of a function f at point x can be viewed as a “spike” or a “bump” at point x with height $f(x)$. If we put a “bump” at every point x we have essentially recovered the function f . The question now boils down to whether a NN can create “bumps”. The answer is affirmative, and a **visual proof** of the “bump” construction and THEOREM 2 is provided by Nielsen 2018.

The universal approximation theorem implies that irrespective of the function we are trying to learn, a network with just one hidden layer and sufficient number of hidden units can represent this function. However, the theorem does not tell us two important things. First, the number of neurons required for the problem we are aiming to solve. Second, *whether we can learn the function at all* (Goodfellow et al. 2016). Learning the true function can fail since the optimizer is not guaranteed to pick the solution that minimizes the generalization loss. Remember it optimizes the training loss as a proxy for the generalization loss.

THEOREM 3 (No free lunch theorem, Wolpert 1996). *Averaged over all possible data generating distributions, every learner has the same error rate when predicting previously unobserved points.*

Finally, we close this section with THEOREM 3. In essence, it states that *no learner is universally better than any other*. Please note that by “learner” we mean even “dummy” learners, such as random guessing. In other words, the more sophisticated learning algorithm we can conceive has the same

¹²In feedforward networks information flows from input to output. In contrast, feedback aka recurrent networks allow the information to travel in both directions by introducing loops. Computations derived from earlier input are fed back into the network, which gives them a kind of memory. This kind of NNs find application in natural language processing (NLP) tasks, such as text generation or classification. Note that although they contain loops, and as such they are not DAGs, we can convert them to DAGs by “unrolling” their computational graph (LeCun et al. 2015).

average performance (over all tasks) as random guessing. While THEOREM 3 seems unintuitive at first glance, it may be easier to understand it with an example. Suppose we see one sheep, and then expect to see another one. We could devise the following strategies (learning algorithms) for predicting the color of the second sheep:

$$\text{strategies} = \left\{ \begin{array}{ll} \text{Same color as the first} & (\text{white, white}) \\ \text{Different color than the first} & (\text{black, black}) \\ \text{Always black} & (\text{white, black}) \\ \text{Always white} & (\text{black, white}) \end{array} \right\} = \text{possible worlds} \quad (2.24)$$

Assuming that all possible worlds (data generating distributions) are *equally likely*, then each strategy has the same expected error: 50%. Fortunately, *in real-world this assumption breaks down*. For example, animals tend to be the same color, so the worlds where the first and the second sheep have different colors are unlikely. In this scenario, guessing the same color as the first is more likely to be correct. Every learning algorithm is equipped with an **inductive bias**, that is a set of assumptions allowing it to *prioritize one hypothesis over another*¹³. Whether algorithm $\mathcal{A}_1$ will outperform $\mathcal{A}_2$ on problem $\mathcal{P}$, is just a matter of who has the right inductive bias.

2.2.2 Regularizing neural networks

Deep NNs typically have hundreds of thousands, million, or even billion parameters. With such a huge number of parameters, they are capable of memorizing a huge variety of complex training sets. However, memorization is harmful from a generalization point of view. Therefore, it is necessary to employ regularization techniques when training deep NNs, especially when the size of the training set is relatively small. Besides adding penalty terms to the training loss as described in SECTION 2.1.3, common regularization techniques include: **data augmentation** and **dropout**.

Data augmentation, as the name implies, is a technique to artificially increase the size of the data set. With this technique, each training instance is replicated many times with *each replicate being randomly distorted in such a way that the corresponding label is left unchanged*¹⁴. For example, in image classification the original images can undergo *geometric transformations* such as rotations, vertical or horizontal flips, small shifts and random crops. Such kind of transformations force the NN to be more tolerant to variations in orientation, position and size. Another way to augment the original images is by applying *color transformations*, such as changing the brightness or contrast of the image, increasing the NN's tolerance in different lighting conditions. Of course, we can further augment our training set by composing geometric and color transformations. The performance boost thanks to data augmentation can be understood in two ways: i). More data is better. ii). As introducing a useful inductive bias. That is, *we know a priori that the true function ought to be invariant in certain transformations, and the augmented images are a way of imposing this knowledge*.

¹³For example, the *composition of layers* in NNs provides a type of relational inductive bias: *hierarchical processing*. Another example of inductive bias is the linear relationship assumed in linear regression.

¹⁴We should be careful on how we augment the training set. For example, in character classification tasks there is difference between the characters "b" $\leftrightarrow$ "d" and "6" $\leftrightarrow$ "9". As such, horizontal flips and 180° rotations are not advisable for this task.

Dropout (Hinton et al. 2012; Srivastava et al. 2014) is a powerful and computational inexpensive method to regularize neural networks, which has proven to be extremely successful. Even the state-of-the-art architectures improved by 1–2 % when dropout was added to them. This may not sound like a lot, but if we consider a model that has already 95 % accuracy, a performance boost of 2 % amounts to 40 % drop in the error rate (going from 5 % to 3 %). Let’s see how it works: at every training step the neurons of a hidden layer can be temporarily “dropped out” with probability p aka *dropout rate*, meaning that they will entirely ignored during this training step, but may become active again in the next training step. And that’s it except for one technical detail. Dropout is applied only during training and as such, all neurons are active during testing. This means that during this phase a neuron will receive a different amount of input signal compared to its (average) input signal during training. For example, if the dropout rate is set to 0.5, then during testing a neuron will be connected to twice as many input neurons as it was during training (on average). To compensate for that, the neuron’s input connections must be multiplied by 0.5 during testing. Otherwise, each neuron in the network will receive a total input signal roughly twice as large as what it was trained on, meaning that each neuron and the network as whole won’t perform well. In general, during testing we need to multiply each input connection by the *keep probability* ($1 - p$). Another way¹⁵ to retain the same amount of input signal for both training and testing phases, is by just dividing each neuron’s output with the keep probability, a technique known as *inverted dropout*.

It is quite surprising that this random “resurrection” of neurons improves the performance of the network. Dropout works because it *forces neurons to pay attention to all of their inputs, rather than relying exclusively on just a few of them, making them robust to the loss of any individual piece of evidence*. Or to put it differently, it strengthens them by forcing them to “live” in a stochastic handicap environment. Another way to understand dropout is the following. When different sets of neurons are “dropped out”, it is like we are training different NNs. That is, the dropout procedure acts as an average of these different networks. The latter will overfit in different ways and so the net effect of dropout will hopefully mitigate this effect.

2.2.3 Convolutional neural networks

Convolutional neural networks are specialized NN architectures to process image-like data and in general, data which exhibit spatio-temporal relationships. They find application in tasks such as text, audio, video and image classification, semantic segmentation and object detection, just to name a few. As their name implies, CNNs make use of the **convolution** operation, so to understand the former we first need to understand the latter.

The convolution between the functions f and g , denoted as $f * g$, is defined as following:

$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau \quad (2.25)$$

which is the integral of the product of f and g when g is reflected about the y -axis and shifted. In CNN terminology the first argument of the convolution operation is referred to as the *input* and the second one as the **kernel** or **filter**. What is the interpretation? It is just a *moving inner product between*

¹⁵Note that these alternative are not perfectly equivalent, but in practice they work equally well.

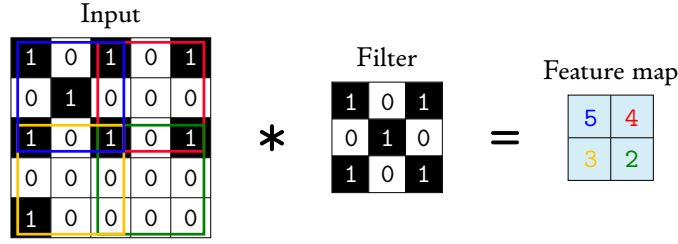


FIGURE 2.9: Convolution operation. Convoluting a filter with an image can be seen as template matching. When a local image patch matches the filter—template to be matched—the output in the feature map is highly positive. Sliding of the filter over the image and recording the output of this template-patch similarity, produces the feature map.

the two functions. Recall that the inner product between two vectors $\mathbf{f}$ and $\mathbf{g}$ is defined as:

$$\mathbf{f} \cdot \mathbf{g} := \sum_i f_i g_i \quad (2.26)$$

and can be viewed as *a measure of similarity* between $\mathbf{f}$ and $\mathbf{g}$. Conceptualizing functions as infinite-dimensional vectors and ignoring the reflection part of convolution¹⁶ let us understand the latter as the inner products between f and shifted versions of g by t . If g represents a pattern—usually local—we are interested in to detect in the signal f , then the output of convolution known as **feature map** in the DL jargon, essentially tell us where (hence the term “map”) the pattern described by g is located in the signal f .

Usually when we work with data on a computer, the index t will be discretized, meaning that it can take only integer values. Assuming that f and g are defined only for integer t , the discrete convolution is defined as following:

$$(f * g)(t) := \sum_{\tau=-\infty}^{\infty} f(\tau)g(t - \tau) \quad (2.27)$$

In ML applications, the input and the filter are usually multidimensional arrays aka *tensors*. Because these two tensors must be explicitly stored separately, we treat the input and the kernel as functions that are zero everywhere except the finite set of points for which their values are stored (Goodfellow et al. 2016).

Please note that we can and often use convolutions over more than one axis at a time. A typical example is when the input is a 2D image. In that case, the convolution between the image I and the filter K is defined as:

$$(I * K)(i, j) = \sum_m \sum_n I(m, n)K(i - m, j - n) \quad (2.28)$$

and is schematically shown in FIGURE 2.9. The reason that the flipping of the filter is not necessary in ML applications, is simply because *the values of the filter are adapted, i.e. learned, during the training*

¹⁶ As it will be latter discussed, whether the kernel is reflected or not, doesn't make a difference for ML applications.

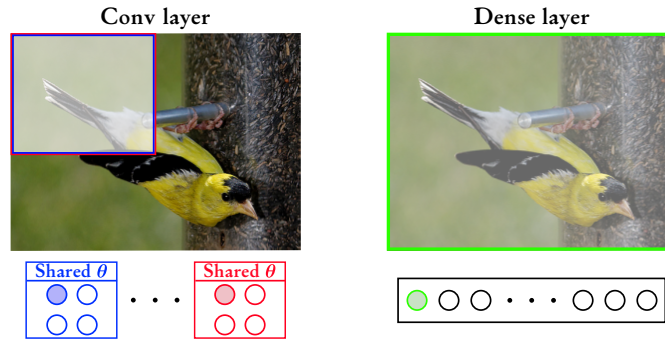


FIGURE 2.10: Neurons' arrangement in convolutional and dense layers. In convolutional layers, neurons are organized into groups and share the same parameters. The receptive field of each member is a small region of the input. In contrast, the receptive field of neurons in a dense layer is the whole input and there is no parameter sharing. Note that the receptive field of neurons incrementally increases as we transition to convolutional layers deeper in the network.

phase. Many DL frameworks instead of the convolution implement a related operation called *cross-correlation*, which is the same as convolution but without flipping the filter. We will stick to this convention for the rest of this section.

Now that the convolution operation has been presented, we can appreciate its contribution to the mechanics of a CNN. *Convolution introduces a beneficial inductive bias to the network, namely **sparse connections** and **parameter sharing***, as shown in FIGURE 2.10, and that's why CNNs outperform FCNNs in object recognition tasks. Sparse connections express the prior knowledge that closely placed pixels are related to each other or to put it differently, *local features such as edges are useful to understand images*. On the other hand, parameter sharing encodes the idea that *a feature detector that is useful in one part of the image is also useful in another part of the image*. Thanks to parameter sharing, once the CNN has learned to recognize a pattern in one location, it can recognize it in any other location. In contrast, once a FCNN has learned to recognize a pattern in one location, it can recognize it only in that particular location (Géron 2017).

The basic building block of a CNN is the *convolutional layer* which contains many *learnable convolutional filters*, each of which is a template that determines whether a particular local pattern is present in an image. It should be emphasized that a feature map of a given layer combines all the feature maps of the previous layer or just the raw image (in the case of the 1-st convolutional layer). This means that if the layers $t - 1$ and t contain n and m feature maps, respectively, then the layer t must learn $m \times n$ filters. By stacking many such layers a CNN extract features hierarchically, with the level of (feature) abstraction increasing the deeper we go into the network.

It is worth to mention that CNNs are essentially regularized FCNNs, meaning that the latter can learn to behave like the former. The catch is that they will probably need a great amount of training data. To understand why this is the case, let's examine the horizontal edge detection problem. The CNN can learn to detect horizontal edges *at any position* by adjusting the values for one of its filters to

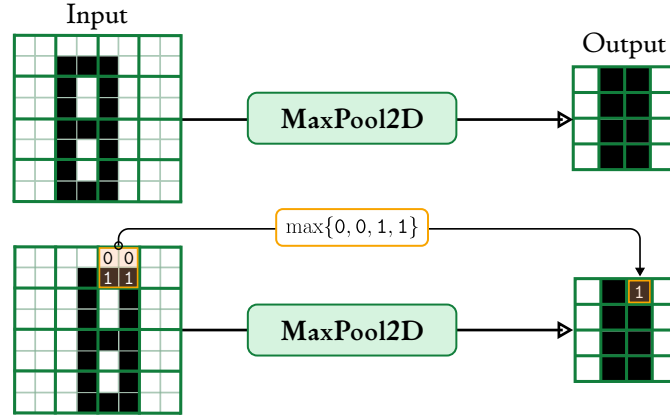


FIGURE 2.II: Max pooling operation. Small translations to the input (input B is just a shifted version of input A by one pixel to the right) produce the same output when passed through the max pooling layer, meaning that the latter introduces into the network some level of invariance to small translations.

the following ones¹⁷:

$$K_x^{\text{edge}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad (2.29)$$

What about the FCNN? FIGURE 2.9 gives the answer. A neuron in a fully connected aka *dense layer* can also learn to detect edges *by just zeroing the weights for its inputs except the small region where the edge needs to be detected*¹⁸. For this region, it must learn the weights specified by EQUATION 2.29. The problem is that the aforementioned zeroing of weights and the non-zero weights themselves, must be learned by many other neurons for horizontal edges to be detected at different positions, which is of course not guaranteed with limited amount of training data.

Besides convolutional layers, another typical of building block of CNNs are the **pooling layers**. Their role is to downsample (reduce the resolution) in a parameter-free way the feature maps produced by convolutional layers. By downsampling in this manner, they reduce the memory-computational footprint of the CNN and also the number of parameters, thereby reducing the risk of overfitting (Géron 2017). A pooling layer takes as inputs the feature maps of the preceding convolutional layer and subsamples them by substituting the outputs in a small neighborhood of the feature map with a summary (Goodfellow et al. 2016). FIGURE 2.II illustrates a common type of pooling, known as **max pooling**, which uses the max function to compute the summary statistic. Another type of pooling is *average pooling*, which computes the summary statistic through averaging. Just like convolution, the idea of pooling generalizes to more than two dimensions.

¹⁷Recall that an “edge” is nothing else than a significant local change in the image intensity. Based on the formula of **symmetric derivative**: $\partial_x f \propto f(x+h, y) + 0 \cdot f(x, y) - f(x-h, y)$. In essence, by convolving an image with the filter specified in EQUATION 2.29, we calculate the gradient along the x -axis in a discretized fashion. Detecting vertical or edges along any direction follows the same idea. You can play with different filters [here](#).

¹⁸We can view each neuron in a dense layer as performing convolution with a kernel size as large as the input.

2.2.4 Training neural networks

Training deep NNs might seem like a nightmare, given their enormous number of parameters. However, modern techniques make it possible to train very deep networks very efficiently. We will start this section by discussing some of the challenges one might face when training deep NNs and then move to describe the optimizers which take the hard job of finding model parameters that hopefully will generalize well. The training of NNs essentially boils down to the following two steps: i). Initialize model parameters ii). Update model parameters.

Therefore, the first question that must be answered is how the parameters must be initialized. First of all, it is important that *all weights are initialized randomly, otherwise training will fail*. For instance, suppose that all weights and biases are initialized to the same constant value, e.g. zero. Then, *all neurons in a given layer will be perfectly identical and thus, any update of the parameters will affect the in exactly the same way*. That is, despite having hundreds or thousands of neurons per layer, the model will end up having “duplicates” of a single neuron in each layer, or to put it differently, *it will act as if it had only one neuron per layer* (Géron 2017). On the contrary, the random initialization of weights *breaks the symmetry* and makes it possible to train a diverse team of neurons. What about the biases? Well, since the asymmetry breaking is already provided by the random initialization of the weights, it is possible and common to initialize the biases to be zero.

It is important that random initialization is performed carefully, otherwise we may end up with **vanishing or exploding gradients**. Since we need the gradients with respect to the parameters in order to update them, if these gradients vanish—i.e. take very small values—then the learning will be very slow. On the other hand, if they explode—i.e. take very large values—then the training either stops due to NaNs or diverges. For simplicity, but without loss of generality consider a NN with N hidden layers and one neuron per layer. The gradient of the loss with respect to the layer l found k steps behind the final hidden layer, equals:

$$\frac{\partial \mathcal{L}_{\text{train}}}{\partial w^l} = \frac{\partial \mathcal{L}_{\text{train}}}{\partial y} \cdot \frac{\partial y}{\partial h^n} \cdot \left(\prod_{i=0}^{k-1} \frac{\partial h^{n-i}}{\partial h^{n-i-1}} \right) \cdot \frac{\partial h^{n-k}}{\partial w^{n-k}} \quad \text{where } n - k = l \quad (2.30)$$

The origin of vanishing and exploding gradients is rooted in the middle term of EQUATION 2.30, which can be expanded to:

$$\prod_{i=0}^{k-1} \frac{\partial h^{n-i}}{\partial h^{n-i-1}} = \prod_{i=0}^{k-1} \phi'(z^{n-i}) w^{n-i} \quad (2.31)$$

The right hand side of EQUATION 2.31 essentially says that the *calculation of gradient involves a series of multiplications*. If the terms involved are all greater than one, then we end up with a very large gradient. In contrast, if all the terms are smaller than one, then the gradient vanishes¹⁹. These effects are more pronounced for earlier layers.

How can we avoid these problems? In essence, we need to find a way to *control the magnitude of this product*. First, notice that EQUATION 2.31 besides the values of weights involves also the derivative

¹⁹ A toy example to understand the problem of unstable gradients is the following. First, replace the activation function $\phi(\cdot)$ with the identity function, i.e. $\phi(z) = z$, then set all weights to the same value w (either smaller or greater than one). In that case, the right hand side of EQUATION 2.31 reduces to w^k which for $w = 1.2$ and $k = 50$ is approximately equal to 9.1×10^4 .

values of the activation function $\phi(\cdot)$. If these are smaller than one, then vanishing gradients are more likely to appear. This is the case when the sigmoid is used as activation function, since the maximum value of its derivative equals 0.25. For this reason, the sigmoid is no longer used as activation function in modern NN architectures. The hyperbolic tangent was used as a replacement but due to its derivative taking values extremely close to zero for large inputs, has given its position to ReLU and its variants. Nevertheless, even if we use ReLU the risk of unstable gradients remains due to the repeated multiplications of weights in EQUATION 2.31. An answer that elegantly accounts for all these subtleties was given by He et al. 2015, who proposed²⁰ an initialization scheme that mitigates the problem of unstable gradients, at least in the early stages of training. For a FCNN with ReLU as activation function this scheme suggests²¹:

$$W_{ij}^l \sim \mathcal{N}\left(0, \frac{2}{n^{l-1}}\right) \quad (2.32)$$

where n^l denotes the size (i.e. number of neurons) of layer l .

Although proper initialization can significantly reduce the risk of unstable gradients at the beginning of the training, it doesn't guarantee that they won't come back later on. The reason is as the training phase proceeds, the parameters keep changing, so the initial control over the product in EQUATION 2.31 is lost. A technique called **batch normalization** (Ioffe et al. 2015) was proposed to address the problem of unstable gradients and its steps are summarized in ALGORITHM 2. A batch normalization layer is inserted between or after the activation function of each hidden layer and it contains two learnable parameters that allow the model to learn the optimal scale and mean of each of the layer's inputs.

ALGORITHM 2: Batch normalization

Input: Values of x_i over mini-batch $\mathcal{B} \subset \mathcal{D}_{\text{train}}$, learnable parameters γ and β

```

/* Mini-batch mean                                     */
1  $\mu_{|\mathcal{B}|} = \frac{1}{|\mathcal{B}|} \sum_i x_i;$ 
/* Mini-batch variance                                   */
2  $\sigma_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_i (x_i - \mu_{\mathcal{B}})^2;$ 
/* Normalize                                             */
3  $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}};$ 
/* Scale and shift                                       */
4  $z_i \leftarrow \gamma \hat{x}_i + \beta;$ 
5 return  $z_i$  for all samples in mini-batch  $\mathcal{B}$ 

```

The authors demonstrated that the addition of batch normalization layers improved all NNs they experimented with, leading to significant improvements in the ImageNet classification task. Moreover, the vanishing gradients problem was significantly reduced, making even possible the use of tanh and sigmoid as activation functions. Adding batch normalization layers decreased the sensitivity to weight initialization and also allowed the use of higher learning rates, accelerating the learning process. Additionally, since the mean and the variance are calculated over mini-batches, batch normalization

²⁰Similar ideas were proposed by Glorot et al. 2010 five years earlier for NNs with tanh as activation function, when the latter was still a common choice.

²¹Similar scheme applies for CNNs and ReLU variants.

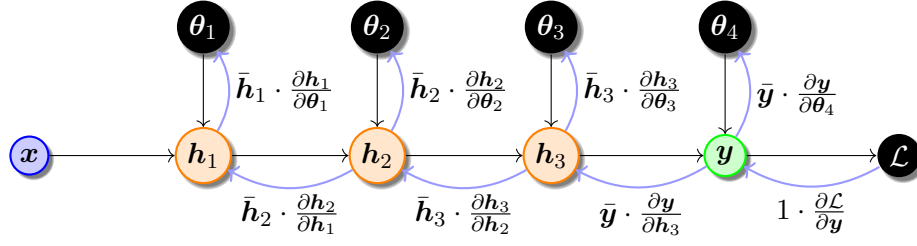


FIGURE 2.12: Illustration of back-propagation. The black and blue arrows show the information flow during the forward and backward pass, respectively. The notation $\bar{z}$ denotes the partial derivative of the loss with respect to the node z . That is, $\bar{z} := \frac{\partial \mathcal{L}}{\partial z}$.

brings a regularization effect due to noisy estimates. It should be noted that during testing the mean and variance are obtained from a running average of the values seen during training.

Training NNs efficiently requires that the calculation of the gradient is extremely fast. A naive approach to calculate the gradient of the training loss with respect to model parameters is based on the definition of the gradient:

$$\frac{\partial \mathcal{L}}{\partial \theta_{ij}} \approx \frac{\mathcal{L}(\theta + h e_{ij}) - \mathcal{L}(\theta)}{h} \quad (2.33)$$

That is, first the training loss for a given parameter configuration θ is calculated by performing a *forward pass* of our training data through the network. Then, we perturb each parameter θ_{ij} by a small amount, perform another forward pass and compare it with the initial loss. The main drawback of this approach is that we need to *perform as many forward passes as the number of model parameters*. Such computational overhead is of course prohibitive for training modern NNs.

Since a NN is ultimately a huge composite function, a more refined approach is to employ the tools of calculus and calculate the derivatives based on the *chain rule*. For a network with 3 hidden layers this amounts to the following calculations:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \theta_4} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \theta_4} \\ \frac{\partial \mathcal{L}}{\partial \theta_3} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{h}_3} \cdot \frac{\partial \mathbf{h}_3}{\partial \theta_3} \\ \frac{\partial \mathcal{L}}{\partial \theta_2} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{h}_3} \cdot \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \cdot \frac{\partial \mathbf{h}_2}{\partial \theta_2} \\ \frac{\partial \mathcal{L}}{\partial \theta_1} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{h}_3} \cdot \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \cdot \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \cdot \frac{\partial \mathbf{h}_1}{\partial \theta_1} \end{aligned} \quad (2.34)$$

From EQUATION 2.34 it is obvious that *some calculations are repeated*. As such, a faster calculation of the gradient is possible by avoiding these repetitions. This is the main idea behind the algorithm called **back-propagation** algorithm, often simply called **backprop**. The procedure for a general computational graph is described in ALGORITHM 3 whereas a schematic representation for the aforementioned NN is provided in FIGURE 2.12. With this algorithm, a single forward and backward pass are enough to obtain the gradient.

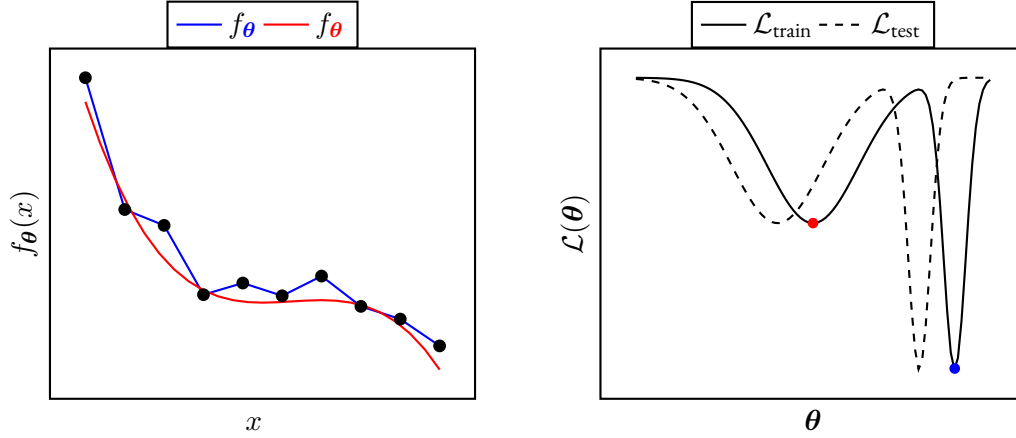


FIGURE 2.13: Machine learning $\neq$ optimization. From a purely optimization perspective, the blue solution is the ideal. In contrast, from a generalization point of view the red solution is preferred since it has lower error on new unseen samples.

ALGORITHM 3: Back-propagation (Rumelhart et al. 1986)

Input: Computational graph $\mathcal{G}$ where nodes u_i follow a topological ordering

```

/* Forward pass */
1 for  $i = 1$  to  $N$  do
2   | Compute  $u_i$  as a function of  $\text{Pa}_{\mathcal{G}}(u_i)$ ;
3 end
4  $u_N = 1$ ;
/* Backward pass */
5 for  $i = N - 1$  to 1 do
6   |  $\bar{u}_i = \sum_{j \in \text{Ch}_{\mathcal{G}}(u_i)} \bar{u}_j \partial_{u_i} u_j$ ;
7 end
8 return derivatives  $\bar{u}_i$ 

```

In the rest of this section, the most common optimizers used for training NNs are presented. All of them are first-order iterative optimization methods, meaning that they can be trapped in local minima of the training loss landscape. However, this is not much of a problem since in ML *the interest is in finding parameters that generalize well, not necessarily the ones that perfectly fits the training data*. As shown in FIGURE 2.13 a local minimum might be a better choice than the global optimum. Moreover, flatter minima should be preferable because they are less sensitive to parameter perturbations or to put it differently, they are less tailored to the specifics of the training set at hand.

Since the training loss is a sum of individual losses (see EQUATION 2.3) and the gradient operator $\nabla(\cdot)$ is linear, the gradient of the training loss with respect to model parameters equals:

$$\nabla_{\theta} \mathcal{L}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{i \in \mathcal{D}_{\text{train}}} \nabla_{\theta} \ell_i(\theta) \quad (2.35)$$

The problem is that *we need to compute all the individual gradients of the training samples*. Typical

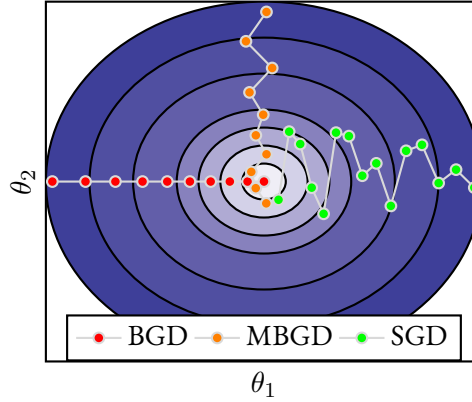


FIGURE 2.14: Variants of gradient descent.

datasets these days contain million training instances, hence it seems wasteful to perform all these calculations for just a single parameter update. Vanilla gradient descent variants come at the rescue by using a only a small subset—aka mini-batch in the DL jargon—of the training set. The insight of these algorithms is that *the gradient over the whole training set is an expectation and as such, it may be approximately estimated using only a small number of training samples*.

The mini-batches are usually drawn from the training set without replacement. These algorithms pass through the training samples until all the training data are used, at which point they start sampling from the full training set again. A single pass through the training set is called **epoch** and the number epochs is the most common criterion for stopping the training of DL algorithms. Depending on the batch size $|\mathcal{B}|$, the gradient descent variants are classified as following²²:

$$\text{variants} = \begin{cases} \text{SGD} & |\mathcal{B}| = 1 \\ \text{MBGD} & 1 < |\mathcal{B}| < |\mathcal{D}_{\text{train}}| \\ \text{BGD} & |\mathcal{B}| = |\mathcal{D}_{\text{train}}| \end{cases} \quad (2.36)$$

The decrease in the computational cost at each iteration comes at the expense of noisy updates as shown in FIGURE 2.14, meaning that these algorithms can't settle at the minimum. Although this is not a major problem, it can be mitigated by increasing the batch size at the last iterations or decrease the learning rate gradually (or both). Usually, the value of the batch size remains constant and only the learning rate changes through the training phase via a *learning rate scheduling* scheme. It should be added that the noisy gradient estimates might be beneficial since they can escape “bad” (e.g. sharp) local minima and saddle points. The aforementioned optimizers are described in ALGORITHM 4.

²²Essentially, the batch gradient descent (BGD) algorithm is the vanilla gradient descent.

ALGORITHM 4: Batch, mini-batch and stochastic gradient descent (Bottou 1998)

Input: $\mathcal{D}_{\text{train}}$, loss function ℓ , model parameters θ , learning rate η , batch size $|\mathcal{B}|$

```

1  $\theta \leftarrow$  random initialization;
2 while stopping criterion not met do
3    $\mathcal{B} \leftarrow$  sample  $|\mathcal{B}|$  datapoints from  $\mathcal{D}_{\text{train}}$ ;
4    $\nabla_{\theta} \mathcal{L}_{\mathcal{B}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta} \ell_i(\theta)$ ;
5    $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}}$ ;
6 end
7 return optimized parameters  $\theta$ 

```

A common modification to the MBGD and SGD algorithms is the addition of a **momentum** term. Instead of using only the gradient of the current step to guide the search, momentum also accumulates the gradient of the previous steps to determine the next update. Momentum keeps track of the previous gradients via an exponentially decaying sum controlled by the hyperparameter $\beta \in (0, 1)$ which is typically set to 0.9. By averaging past gradients, their zig-zag directions (see FIGURE 2.14) effectively cancel out, leading to a smoother trajectory. Note that the value of β essentially controls how quickly the effect of the past gradients decay.

ALGORITHM 5: Momentum (Polyak 1964)

Input: Model parameters θ , momentum β , learning rate η

```

1  $\theta \leftarrow$  random initialization;
2 while stopping criterion not met do
3    $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}}$ ;
4    $\theta \leftarrow \theta + \mathbf{m}$ ;
5 end
6 return optimized parameters  $\theta$ 

```

Another commonly used optimizer for training NNs is the Adam optimizer, which is described in ALGORITHM 6. Besides the momentum term²³ $\mathbf{m}$ the update is controlled also by the term $\mathbf{s}$ which keeps track the square²⁴ of the past gradients. This term provides the means for using *adaptive learning rates* which can speed up convergence by pointing the resulting updates more towards the local minimum. The steps 5 and 6 are somewhat of a technical detail: since $\mathbf{m}$ and $\mathbf{s}$ are both initialized at $\mathbf{0}$, they will be biased towards $\mathbf{0}$ at the first iterations. These steps just boost $\mathbf{m}$ and $\mathbf{s}$ at the beginning of the training (Géron 2017). Typical values for the hyperparameters β_1 and β_2 are 0.9 and 0.999, respectively. Please note that the symbol $\odot$ represents the element-wise multiplication whereas $\oslash$ represents the element-wise division.

²³Compared to momentum, Adam computes an exponentially decaying average rather than an exponentially decaying sum but these are actually equivalent except for a constant factor.

²⁴That is, each s_i accumulates the squares of the partial derivative $\frac{\partial \mathcal{L}}{\partial \theta_i}$.

ALGORITHM 6: Adam (Kingma et al. 2017)

Input: Model parameters θ , learning rate η , smoothing term ϵ , momentum decay β_1 , scaling decay β_2

- 1 $\theta \leftarrow$ random initialization;
- 2 **while** *stopping criterion not met* **do**
- 3 $\mathbf{m} \leftarrow \beta_1 \mathbf{m} - (1 - \beta_1) \nabla_{\theta} \mathcal{L}_{\mathcal{B}}$;
- 4 $\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\theta} \mathcal{L}_{\mathcal{B}} \odot \nabla_{\theta} \mathcal{L}_{\mathcal{B}}$;
- 5 $\widehat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^t}$;
- 6 $\widehat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t}$;
- 7 $\theta \leftarrow \theta + \eta \widehat{\mathbf{m}} \oslash \sqrt{\widehat{\mathbf{s}} + \epsilon}$;
- 8 **end**
- 9 **return** *optimized parameters* θ

Chapter 3

Methodology



NEURAL NETWORKS are notorious for being “data hungry”, requiring a relatively large amount of training data, in order to unleash their full potential. As such, to get a representative picture of the capabilities of the proposed DL framework, two large datasets are employed to train the 3D CNN. The first one, is a subset of the University of Ottawa (UO) database (Boyd et al. 2019), and is used to verify the applicability of the proposed pipeline, examining CO₂ uptake. The second one, is the COFs database generated by (Mercado et al. 2018), and is employed to demonstrate the transferability of the approach, examining CH₄ uptake. Please note, that these datasets are already labeled, and as such no molecular simulations were performed in this study to generate the labels (gas uptakes) of the materials. Information regarding the Grand Canonical Monte Carlo (GCMC) calculations that were performed to produce the labels, can be found in the original works.

3.1 Datasets

3.1.1 MOFs dataset

The UO database is composed of 324 426 hypothetical MOFs. Randomly selected subsets of size 32 432, 5000 and 27 438, served as the training, validation[†] and test sets (see SECTION 2.1.4), respectively. The absolute CO₂ uptake at 298 K and 0.15 bar was examined and the following eight textual properties were used as input for the conventional models: unit cell’s mass and volume, gravimetric surface area, void fraction, void volume, largest free sphere diameter, largest included sphere along free sphere path diameter and largest included sphere diameter. For producing the learning curves shown in FIGURE 4.3a, the training set size was varied and the following training set sizes were considered:

$$\{100, 500, 1000, 2000, 5000, 10\,000, 15\,000, 20\,000, 32\,432\} \quad (3.1)$$

The energy voxels of MOFs are publicly available in [figshare](#).

3.1.2 COFs dataset

The COFs database contains 69 839 and provides data for five textual properties and CH₄ uptake at different thermodynamic conditions. A randomly selected subset of 55 871 materials served as the training set, whereas the remaining 13 968 correspond to the test set. In this work, CH₄ uptake at 298 K and

[†]The validation set was used to select the number of epochs, see SECTION 3.3.2.

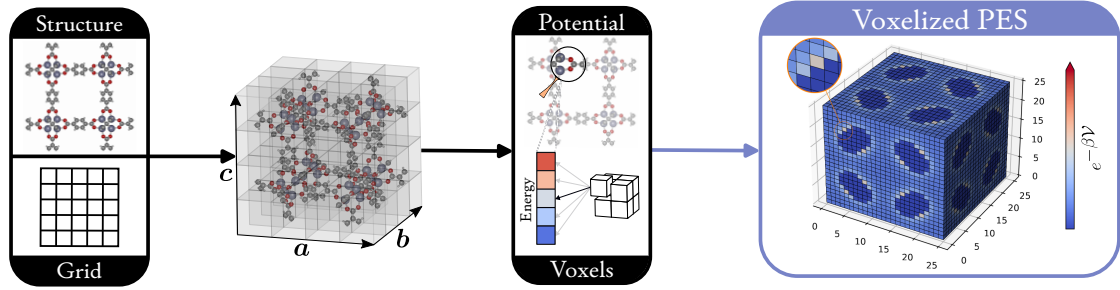


FIGURE 3.1: Workflow to construct the voxelized PES. The grid size and the type of the potential control the “trade-off” between information content and computational cost. The IRMOF-1 structure was visualized with the iRASP software (Dubbeldam et al. 2018).

5.8 bar was examined. The following five textual properties were used to build the conventional models: density, gravimetric surface area, void fraction, pore limiting diameter and largest cavity diameter. For producing the learning curves shown in FIGURE 4.3b, the training set size was varied and the following training set sizes were considered:

$$\{5000, 10\,000, 15\,000, 20\,000, 35\,000, 55\,871\} \quad (3.2)$$

3.2 Voxelized PES

In order to calculate the voxelized PES, first a 3D grid of size $n \times n \times n$ is overlayed over the unit cell of the material. Second, at each voxel centered at grid point $\mathbf{r}_i$, the interaction of the guest molecule with the framework atoms $\mathcal{V}(\mathbf{r}_i)$ is calculated, and this energy value “colorizes” the corresponding voxel. The workflow to construct the voxelized PES is schematically depicted in FIGURE 3.1. The grid size n and the type of the potential control the “trade-off” between information content and computational cost. The greater the grid size n , the greater the resolution of the energy image and as such, the information content. However, this comes at the cost of increased computational cost which is by no means negligible, since voxelization scales up as $\mathcal{O}(n^3)$. Similarly, the more accurate the modeling of interactions, the greater the information content, but again, extra computational burden is required. *The voxelized PES converges to the exact one as $n \rightarrow \infty$ and when the voxels are filled with energy values derived from ab-initio calculations.*

In this work we strived for minimal computational cost, setting $n = 25$ and modeling all interactions with the Lennard-Jones (LJ) potential, using a spherical probe molecule as guest. The interaction energy $\mathcal{V}(\mathbf{r}_i)$ between the spherical probe and the framework atoms was calculated as following:

$$\mathcal{V}(\mathbf{r}_i) = \sum_{\substack{j=1 \\ r_{ij} \leq r_c}}^N 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] \quad (3.3)$$

where N is the number of framework atoms, r_c is the cutoff radius which was set to 10 Å, r_{ij} is the distance between the j -th framework atom and the probe molecule and ϵ_{ij} and σ_{ij} combine the ϵ and



σ values of the probe molecule and the j -th framework atom using the Lorentz-Berthelot mixing rules:

$$\sigma_{ij} = \frac{\sigma_i + \sigma_j}{2} \quad \wedge \quad \epsilon_{ij} = \sqrt{\epsilon_i + \epsilon_j} \quad (3.4)$$

If there is geometric overlap between a grid point and the position of a framework atom, the interaction energy can be extremely repulsive, leading to very large, even infinite values, which can hamper or not allow the training of a NN at all. For this reason, each voxel was filled with $e^{-\beta \mathcal{V}(\mathbf{r}_i)}$, which tends to 0 as $\mathcal{V}(\mathbf{r}_i) \rightarrow \infty$, where $\beta = \frac{1}{k_B T}$ is the Boltzmann constant and T is the temperature, which was set at 298 K. The Python package **MOX ϵ** was introduced to facilitate and speed through parallelization the calculation of energy voxels. In the remaining of this thesis, the terms “voxelized PES” and “energy voxels”, are used interchangeably.

3.3 Machine Learning Details

For the conventional ML models, the RF algorithm as implement in the scikit-learn (Pedregosa et al. 2011) package (version 1.2.2) was used, while the PyTorch (Paszke et al. 2019) framework (version 2.0.1+cu118) was employed for the CNN models. The performance, i.e. the generalization ability of the models, was assessed by the coefficient of determination R^2 :

$$R^2 := 1 - \frac{\sum_{i=1}^{N_{\text{test}}} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N_{\text{test}}} (y_i - \bar{y})^2} \quad (3.5)$$

where N_{test} is the number of samples in test set, $\bar{y}$ is the mean value of y in the test set and $y_i, \hat{y}_i$ are the ground truth and predicted values of the i -th sample, respectively. In all cases where confidence interval (CI) are presented, they were calculated using the percentile bootstrap method (Efron et al. 1994), with 10 000 bootstrapped samples from the test set.

3.3.1 CNN architecture

The architecture of the 3D CNN is presented in FIGURE 4.1, whereas a PyTorch implementation is publicly available in: **RetNet**. Kernel size is set to 3 for Conv1, Conv2 and 2 for Conv3, Conv4 and Conv5 layers. Stride equals 1 for all convolutional layers and only Conv1 layer is padded, with “same” padding and “periodic” mode. For both MaxPool layers, kernel size and stride are both set to 2. For the Dropout layer (see also SECTION 2.2.2), the dropout rate p equals 0.3, while the negative slope is set to 0.01 for all LeakyReLU layers.

3.3.2 Preprocessing & CNN training details

Prior to entering the CNN the energy voxels are standardized “on the fly” based on the training set statistics—this transformation is applied both during training and inference—which are computed channel wise². The voxelized PES of a material $\mathbf{x}$, enters the CNN as following:

$$\mathbf{x}' = \frac{\mathbf{x} - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (3.6)$$

²The voxelized PES is essentially a single channel, i.e. grayscale, image.



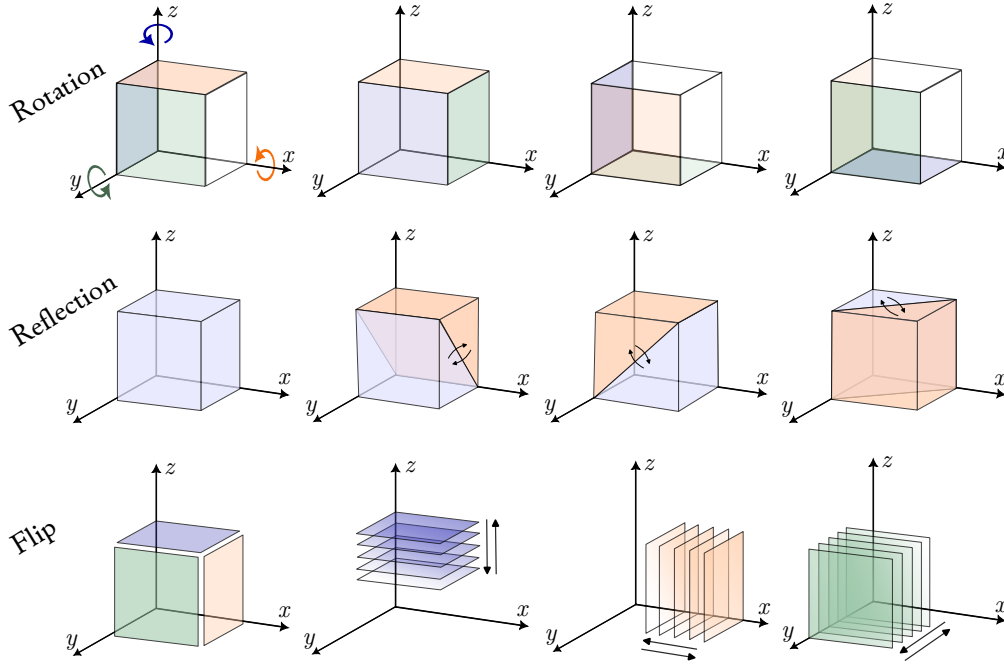


FIGURE 3.2: Geometric transformations for data augmentation.

Regarding CNN training, MSL is used as loss function and weights are initialized according to the He scheme (He et al. 2015). The Adam optimizer (Kingma et al. 2017) is employed, with $|\mathcal{B}| = 64$, $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1 \times 10^{-8}$. The CNN training lasts for 50 epochs with the learning rate being decayed by 0.5 every 10 epochs. RetNet was trained in the MOFs dataset with the largest training set size (32 432 training samples), for a different number of epochs, namely 10, 20 and 50. The latter value was selected, since it showed the greatest performance in the validation set.

3.3.3 Data augmentation

With this technique, the training set is artificially increased, by applying transformations on the input that leave the label unchanged (see also SECTION 2.2.2). With regards to gas adsorption, this amounts to applying geometric transformations on the voxelized PES, that leave the gas uptake value of the material unchanged. Data augmentation, helps the CNN to combat overfitting—e.g. memorizing specific orientations of the voxelized PES—and focus on the underlying patterns.

In this work, four types of geometric transformations are applied (including the identity one), as shown in FIGURE 3.2. At each training iteration, the samples in the batch undergo one of these transformations, with all transformations having the same probability to be applied. For instance, at one training iteration, the voxelized PES can be rotated 90° around the x -axis, while at another iteration, it might be flipped along the z -axis. Rotation is performed either clockwise or counterclockwise, around one of the three axes. The voxelized PES can also be viewed as a stack of 2D slices. In this view, reflection corresponds to transposing each slice, whereas flip reversed the order of the slices. Reflection takes place along one of the xy , xz , yz planes, whereas flip is performed along one of the three axes. FIGURE 3.3 illustrates the performance difference when the CNN is trained on the MOFs dataset with and

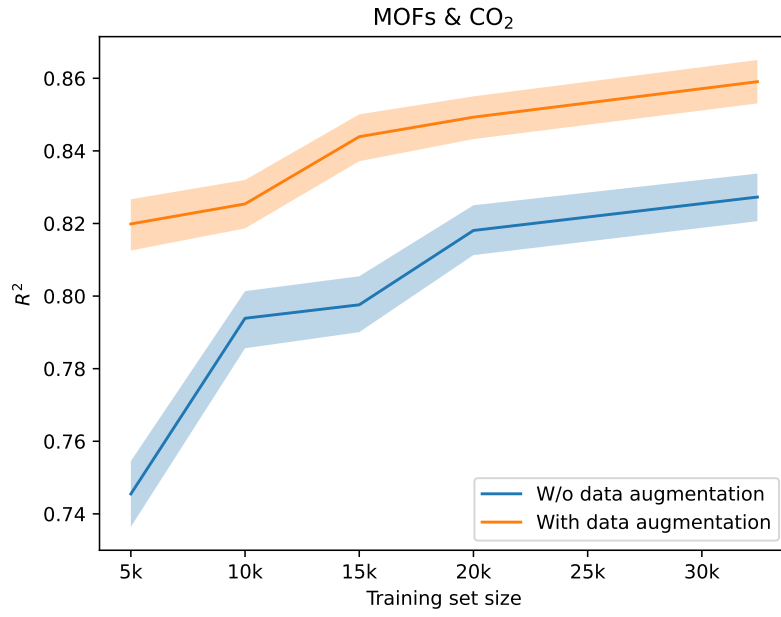


FIGURE 3.3: CNN performance (R^2 score) on test set with and without data augmentation. Shaded areas correspond to the 95 % CI.

without data augmentation, for training set sizes:

$$\{5000, 10\,000, 15\,000, 20\,000, 32\,432\} \quad (3.7)$$

Chapter 4

Results & Discussion



THE PROPOSED FRAMEWORK is initially tested on the UO database, for predicting CO_2 uptake in MOFs, the gas that mainly “triggered” the development of energy-based descriptors. In order to evaluate the transferability of the approach, a different host-guest system is also examined. We apply the suggested approach in the database created by Mercado et al. (2018), for predicting CH_4 uptake in COFs. In both cases, the resulting ML models are compared with conventional ones, built upon geometric descriptors. In the rest of this chapter, results from these comparisons are presented, followed by discussion for improvements of the proposed framework. Before delving into the results, we first take a look at RetNet, the 3D CNN under the hood, that takes as input a voxelized PES and outputs a prediction for a gas adsorption property, hereon gas uptake.

4.1 Visualizing RetNet

FIGURE 4.1 illustrates the processing a voxelized PES undergoes, as it is passing through RetNet. For the purpose of this visualization, we use the model trained on the MOFs dataset with the largest training set size (see SECTION 3.1). Moreover, for the ease of visualization, only some feature maps (see SECTION 2.2.3) of RetNet are visualized. Please note, that each feature map of a given layer, combines all the feature maps of the precedent layer. The only exception are the pooling layers, which just downsample the feature maps from the previous layers.

For example, each feature map of the Conv2 layer takes into account all the 12 feature maps of Conv1 layer. In contrast, the feature maps of the MaxPool1 layer, are just downsampled versions of the corresponding feature maps in Conv2 layer. Although feature maps of CNNs are not meant to be interpreted by humans—especially the ones found deeper in the network—it is worth noticing that early convolutional layers (i.e. Conv1 and Conv2) emphasize the texture of the structure. For instance, the 3rd feature map of Conv1 layer delineates the skeleton of the framework.

Moving towards the output layer, the alternation of max pooling and convolutional layers continues until the flatten layer, which just flattens out and concatenates¹ all feature maps from Conv2 layer into a single vector of size 3240. This vector is then processed by a FCNN—i.e. the stack of dense and output layers—to give the final prediction. Since the output layer is really nothing more than a linear

¹Given m feature maps of size $n \times n \times n$, a flatten layer converts them into a vector of size mn^3 .

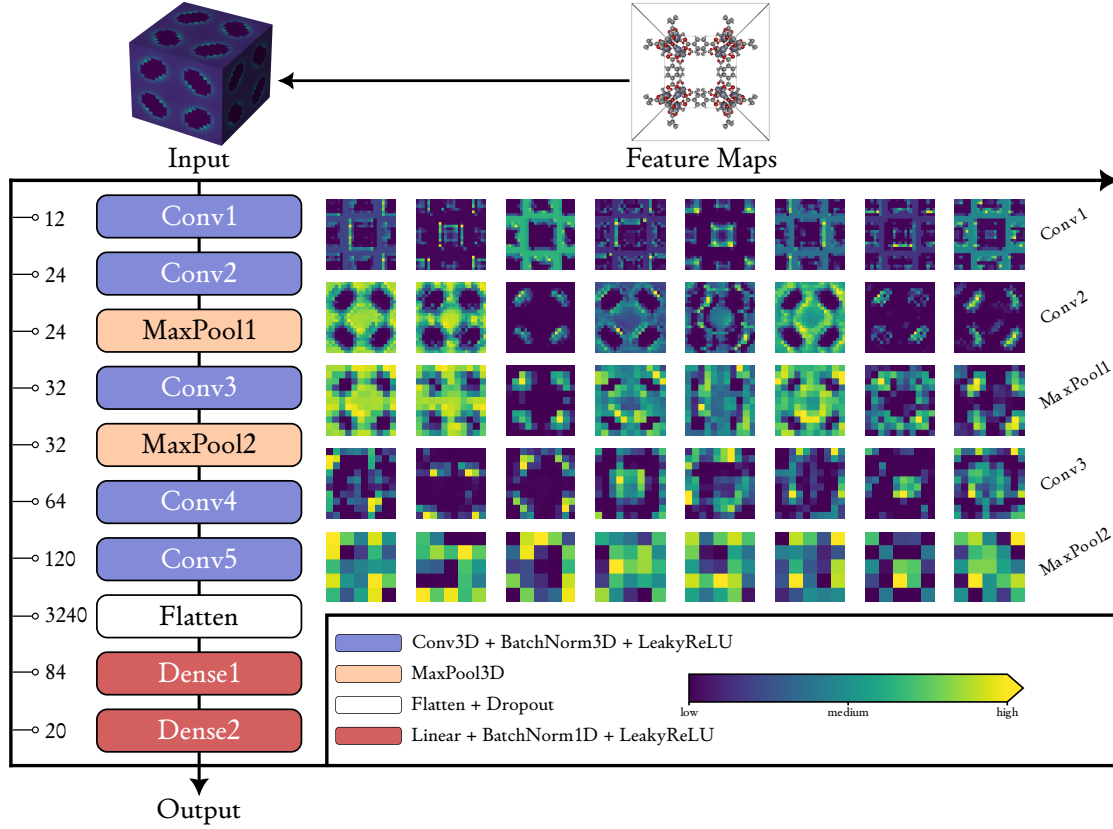


FIGURE 4.1: Forward pass of IRMOF-1 through RetNet. For the sake of visualization, only slices (feature maps are 3D matrices) of 8 feature maps from the first 5 layers are visualized. For Conv1 layer, the 5-th slice is presented, while for the remaining layers, the 1-st slice is presented. The IRMOF-1 structure was visualized with the iRASPA software (Dubbeldam et al. 2018).

layer (see SECTION 2.2.1), all that RetNet does is the following:

$$\underbrace{\text{PES}}_{\text{input}} \mathbf{x} \longrightarrow \underbrace{\text{fingerprint}}_{\text{feature extraction}} \phi(\mathbf{x}; \boldsymbol{\theta}) \longrightarrow \underbrace{\text{gas uptake}}_{\text{output}} \beta^T \phi(\mathbf{x}; \boldsymbol{\theta}) + \beta_0 \quad (4.1)$$

EQUATION 4.1 says that RetNet, starting from the PES, extracts a fingerprint—that is, a high level representation of the PES—and then predicts the gas uptake by using a linear model on top of this fingerprint. All intermediate layers between input and output layer participate in this feature extraction step, with the Dense2 layer determining the size of the fingerprint, which is a vector of size 20, i.e. $\phi(\mathbf{x}) \in \mathbb{R}^{20}$ (see FIGURE 4.2). The fact that *this fingerprint extraction step is learnable*—the parameters $\boldsymbol{\theta}$ of ϕ are learned during the training phase—is what fundamentally distinguishes the proposed approach from methods that use hand-crafted fingerprints (see SECTION 1.3). In these methods the fingerprint or extraction step is fixed, and based on some heuristic, such as energy histograms (Bucior,

Bobbitt, et al. 2019) or average interactions (Fanourgakis, Gkagkas, Tylianakis, and Froudakis 2020). Hereon, feature extraction from the PES is no longer fixed, but is an essential part of the training phase.

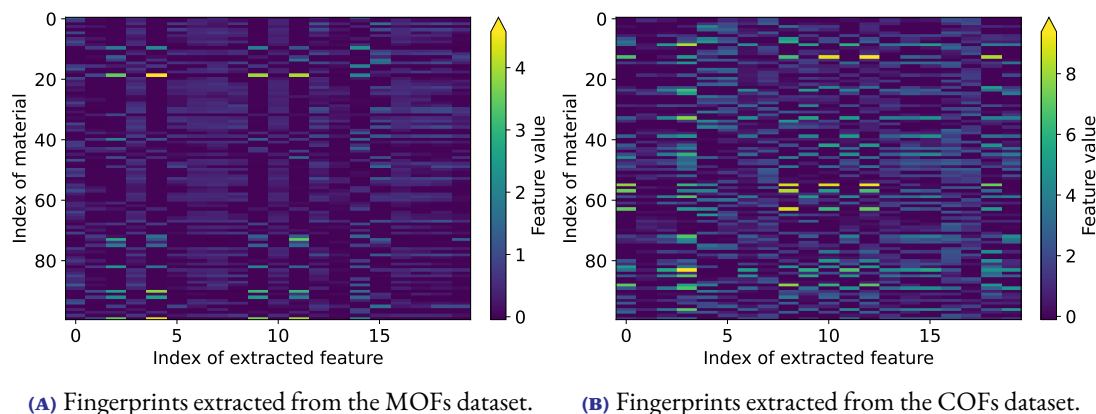


FIGURE 4.2: Output of the last LeakyReLU layer of RetNet trained on MOFs (left) and COFs (right) datasets, with the corresponding maximum training set size. The fingerprints of the first 100 materials in the training set are depicted.

4.2 Learning Curves & Parity Plots

The learning curves of the conventional models—built upon geometric descriptors—and the proposed ones—built upon energy voxels—are shown in FIGURE 4.3. As it can be seen from FIGURE 4.3a, in the MOFs-CO₂ case, the CNN model achieves an R^2 score of 0.859, outperforming the conventional model, which shows an R^2 score of 0.690. This amounts to around 25 % increase in accuracy, even with such a coarse approximation of the PES². Moreover, from the same figure, one can notice that the proposed model reaches the peak performance of the conventional one—that is, the performance when trained with the maximum training set size—by requiring two orders of magnitude less training data, around 300.

Analogous results are observed when examining the COFs-CH₄ case. Again the CNN model performs better, showing an R^2 score 0.969 compared to 0.941 for the conventional one. Similar to the previous case, a substantially smaller amount of training data are required—one order of magnitude less training, around 6900—for the CNN model to match the performance of the conventional model.

The fact that in both cases, the learning curves of the proposed models lie above the corresponding ones of the conventional models, should be credited to the following factors. First, the increased informativeness of the voxelized the voxelized PES in comparison to geometric descriptors. Second, the ability of CNNs to handle images and image-like data, such as the voxelized PES, which is essentially a single channel 3D image. And last but not least, the data augmentation technique, which was applied during the CNN training (see SECTION 3.3.3).

A parity plot is a scatter plot that compares a set of predictions from a ML model against a set of

²In this work, all host-guest interactions were modeled with the LJ potential (see SECTION 3.2), which neglects electrostatic interactions.

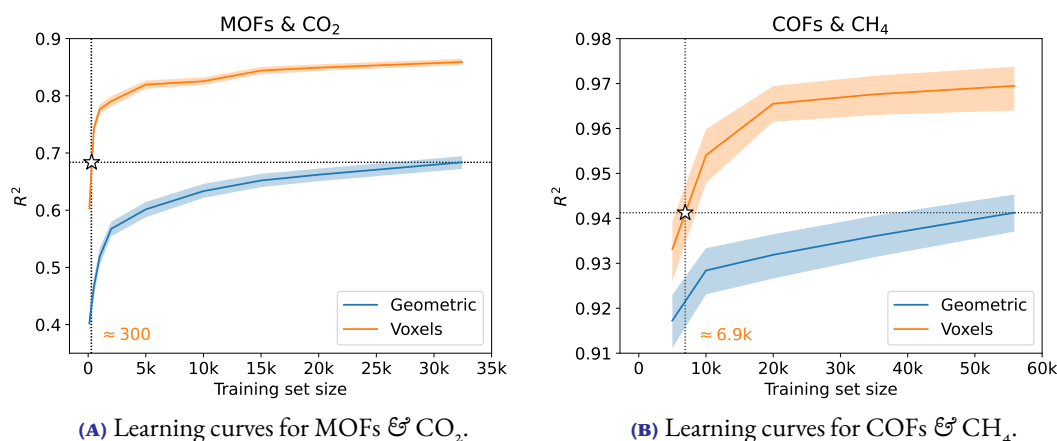


FIGURE 4.3: Performance (R^2 score) on test set as function of the training set size for conventional and CNN models. Shaded areas correspond to the 95 % CI. The x -coordinate of the white star denotes the training set size where the CNN model reaches the performance of the conventional one, the y -coordinate. “Geometric” stands for geometric descriptors, while “Voxels” stands for energy voxels.

benchmark (ground truth) values³. Each point has (x, y) coordinates, where x is the ground truth value and y is the corresponding (predicted) value from the model. For a perfect model, all the points should lie on the diagonal. From the parity plots of FIGURE 4.4, it can be seen that the CNN produces more reliable predictions, since more points are closer to the diagonal. Specifically, for the region of high-performing materials the conventional model significantly underestimates their performance, while the CNN model “pushes” its predictions closer to the diagonal.

4.3 Discussion

It is worth mentioning the increase in performance, approximately 13 %, of the CNN model in the COFs- CH_4 case ($R^2 = 0.969$) compared to the MOFs- CO_2 case ($R^2 = 0.859$). In contrast to CO_2 , which exhibits strong electrostatic interactions with the framework atoms, CH_4 lacks dipole or quadrupole moment. Given that the same resolution—i.e. the same grid size—was used in both cases and that the LJ potential doesn’t account for electrostatic interactions, this performance gap should be attributed to the absence of the latter in the voxelized PES. *In other words, the extra “contrast” that such strong interactions add to the energy image of the material, is missing from the voxelized PES. As such, a straightforward approach to improve the performance of the proposed approach, especially for adsorbates like CO_2 , H_2 and H_2S , is to include this type of interactions into the voxelized PES.* Of course, there is no free lunch, since these refinements require the assignment of partial charges to each framework atom, which is a computationally expensive task. Luckily, ML-based approaches have already been developed (Bleiziffer et al. 2018; Raza et al. 2020; Kanchalapalli et al. 2021), which can assign partial charges rapidly and with high fidelity, enabling the efficient construction of a more accurate voxelized PES.

³Most of the time (and also in this work), the ground truth values are obtained from molecular simulations.

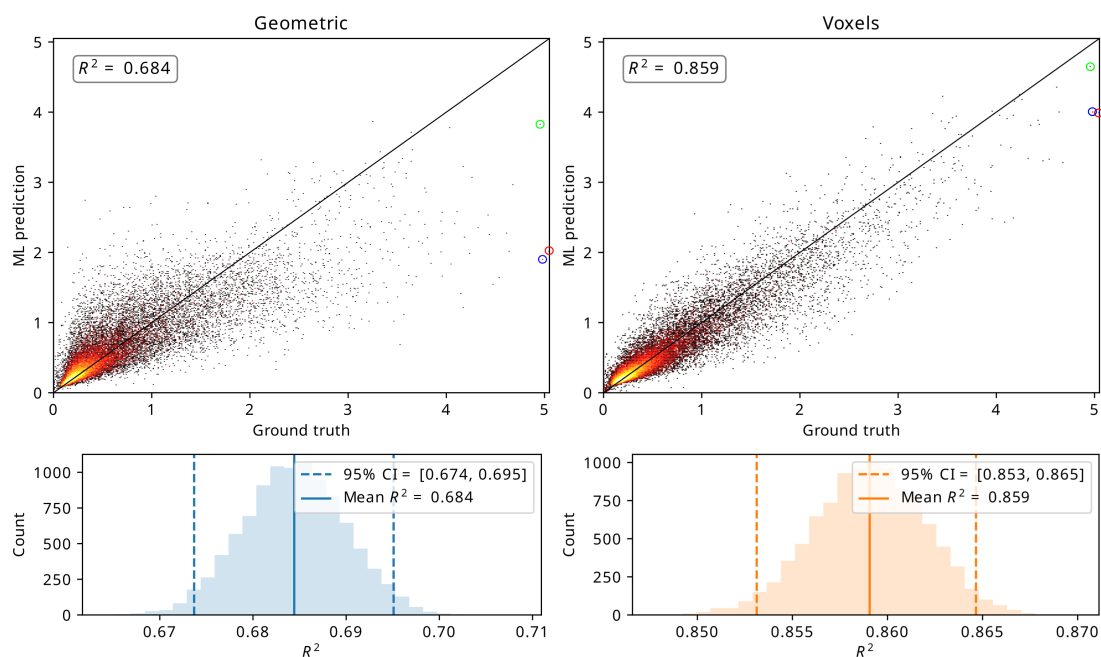


FIGURE 4.4: Parity plots (top) for the ML models with geometric descriptors (RF model) and energy voxels (CNN model) regarding CO_2 uptake in MOFs. Histograms (bottom) of the R^2 values used to construct the 95 % CIs. Both models were trained with the largest training set (32 432 training samples). “Geometric” stands for geometric descriptors while “Voxels” stands for energy voxels. Ground truth values are the theoretical values obtained from the UO database (measured in mmol g^{-1}). Circled points in green, blue and red, correspond to the top-1, top-2 and top-3 materials, respectively.

Improving the input, and as such, the performance of the suggested pipeline is a major concern, but not the only one. *What about the data efficiency of the pipeline?* Imagine that we are asked to predict CH_4 uptake at various thermodynamic conditions. A naive approach would be to collect training data and retrain from scratch the CNN for every thermodynamic condition, which is of course a laborious task. *Can we do something smarter?* Well, the fact that the proposed framework uses a DL algorithm under the hood, opens the door for applying **transfer learning techniques**. In a nutshell, transfer learning (Zhuang et al. 2019; R. Ma et al. 2020; Kang et al. 2023) is based on the following idea: *a violist can learn to play piano faster than others, since both the piano and the violin are musical instruments, and may share some common knowledge*. Translating this to NNs, a pre-trained NN on an original task—known as the *source task*—may require less training data to perform well on a new task—known as the *target task*—if there is some *similarity between the tasks*. Coming back to our “imaginary” scenario, all we have to do is to train the CNN once in a specific thermodynamic condition⁴ and then fine-tune this pre-trained model on the other conditions.

Throughout this work we focused on gas adsorption, but of course this doesn’t mean we are not interested in predicting other properties of reticular materials. *What if we are asked to predict properties such as band gap or bulk modulus?* In that case, quantities such as *electron density* are more informative over host-guest interactions with regards to the aforementioned properties. This entails that the *voxelized electron density* should substitute the voxelized PES, as input to the 3D CNN. Nevertheless, **wouldn’t be great if all properties could be predicted from one and only one input?** If our aim is to predict *different properties for the same structure, shouldn’t the structure itself be used as input?* Currently, the approaches to tackle this challenge are based on *text representations* (Bucior, Rosen, et al. 2019; Cao et al. 2023) and *crystal graphs* (Xie et al. 2018; Chen et al. 2019). The main drawback of these approaches, is their inability to represent exactly the structure, that is the *exact arrangement of the atoms in the 3D space*.

Point clouds (Qi et al. 2016; Bello et al. 2020) are a natural way to solve this problem, since they are just *a set of coordinates and associated features*. In our context, the coordinates are the coordinates of the atoms, and the associated features are the types of the atoms. It should be emphasized, that *a point cloud is not another mathematical representation of a material—in the sense of a descriptor—it is the material itself*⁵. Therefore, an answer to the original question is to *couple point clouds with a neural network that can handle such kind of input*. This approach might have to overcome the current immaturity of DL over point clouds—especially regarding materials and molecules—but from a chemical perspective, it is the only one that truly respects the 3D nature of chemistry and of course, reticular chemistry.

⁴Preferably, the one where we have more labeled training data.

⁵Same ideas apply for molecules and in general, for any chemical system.

Bibliography

- [1] F. Rosenblatt. *The Perceptron, a Perceiving and Recognizing Automaton Project Para*. Report: Cornell Aeronautical Laboratory. Cornell Aeronautical Laboratory, 1957.
- [2] B.T. Polyak. “Some methods of speeding up the convergence of iteration methods”. In: *USSR Computational Mathematics and Mathematical Physics* 4.5 (1964), pp. 1–17. DOI: [https://doi.org/10.1016/0041-5553\(64\)90137-5](https://doi.org/10.1016/0041-5553(64)90137-5).
- [3] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (Oct. 1986), pp. 533–536. DOI: [10.1038/323533a0](https://doi.org/10.1038/323533a0).
- [4] K. Hornik, M. Stinchcombe, and H. White. “Multilayer feedforward networks are universal approximators”. In: *Neural Netw.* 2.5 (June 1989), pp. 359–366.
- [5] B. Efron and R.J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC Monographs on Statistics & Applied Probability. Taylor & Francis, 1994.
- [6] David H. Wolpert. “The Lack of A Priori Distinctions Between Learning Algorithms”. In: *Neural Computation* 8.7 (Oct. 1996), pp. 1341–1390. DOI: [10.1162/neco.1996.8.7.1341](https://doi.org/10.1162/neco.1996.8.7.1341).
- [7] T.M. Mitchell. *Machine Learning*. McGraw-Hill International Editions. McGraw-Hill, 1997.
- [8] Léon Bottou. “Online Algorithms and Stochastic Approximations”. In: *Online Learning and Neural Networks*. Ed. by David Saad. revised, oct 2012. Cambridge, UK: Cambridge University Press, 1998.
- [9] Nathaniel L. Rosi et al. “Hydrogen Storage in Microporous Metal-Organic Frameworks”. In: *Science* 300.5622 (2003), pp. 1127–1129. DOI: [10.1126/science.1083440](https://doi.org/10.1126/science.1083440).
- [10] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Berlin, Heidelberg: Springer-Verlag, 2006.
- [11] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. 1st ed. Springer, 2007.
- [12] Yingwei Li and Ralph Yang. “Gas Adsorption and Storage in Metal-Organic Framework MOF-177”. In: *Langmuir* 23.26 (Nov. 2007), pp. 12937–12944. DOI: [10.1021/la702466d](https://doi.org/10.1021/la702466d).
- [13] Shengqian Ma et al. “Metal-Organic Framework from an Anthracene Derivative Containing Nanoscopic Cages Exhibiting High Methane Uptake”. In: *Journal of the American Chemical Society* 130.3 (Dec. 2007), pp. 1012–1016. DOI: [10.1021/ja0771639](https://doi.org/10.1021/ja0771639).
- [14] Jihyun An, Steven J. Geib, and Nathaniel L. Rosi. “High and Selective CO₂ Uptake in a Cobalt Adeninate Metal-Organic Framework Exhibiting Pyrimidine- and Amino-Decorated Pores”. In: *Journal of the American Chemical Society* 132.1 (Dec. 2009), pp. 38–39. DOI: [10.1021/ja909169x](https://doi.org/10.1021/ja909169x).

- [15] Jia Deng et al. "ImageNet: A large-scale hierarchical image database". In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, June 2009. DOI: [10.1109/cvpr.2009.5206848](https://doi.org/10.1109/cvpr.2009.5206848).
- [16] Hiroyasu Furukawa and Omar M. Yaghi. "Storage of Hydrogen, Methane, and Carbon Dioxide in Highly Porous Covalent Organic Frameworks for Clean Energy Applications". In: *Journal of the American Chemical Society* 131.25 (2009), pp. 8875–8883. DOI: [10.1021/ja9015765](https://doi.org/10.1021/ja9015765).
- [17] T. Hastie, R. Tibshirani, and J.H. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer series in statistics. Springer, 2009.
- [18] Jesse Read et al. "Classifier Chains for Multi-label Classification". In: *Machine Learning and Knowledge Discovery in Databases*. Springer Berlin Heidelberg, 2009, pp. 254–269. DOI: [10.1007/978-3-642-04174-7_17](https://doi.org/10.1007/978-3-642-04174-7_17).
- [19] Xavier Glorot and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Yee Whye Teh and Mike Titterton. Vol. 9. Proceedings of Machine Learning Research. Chia Laguna Resort, Sardinia, Italy: PMLR, May 2010, pp. 249–256.
- [20] Jose L. Mendoza-Cortes, Tod A. Pascal, and William A. Goddard. "Design of Covalent Organic Frameworks for Methane Storage". In: *The Journal of Physical Chemistry A* 115.47 (Nov. 2011), pp. 13852–13857. DOI: [10.1021/jp209541e](https://doi.org/10.1021/jp209541e).
- [21] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [22] Myunghyun Paik Suh et al. "Hydrogen Storage in Metal-Organic Frameworks". In: *Chemical Reviews* 112.2 (Dec. 2011), pp. 782–835. DOI: [10.1021/cr200274s](https://doi.org/10.1021/cr200274s).
- [23] Kenji Sumida et al. "Carbon Dioxide Capture in Metal-Organic Frameworks". In: *Chemical Reviews* 112.2 (Dec. 2011), pp. 724–781. DOI: [10.1021/cr2003272](https://doi.org/10.1021/cr2003272).
- [24] Christopher E. Wilmer et al. "Large-scale screening of hypothetical metal-organic frameworks." In: *Nature chemistry* 4 2 (2011), pp. 83–9.
- [25] Omar K. Farha et al. "Metal-Organic Framework Materials with Ultrahigh Surface Areas: Is the Sky the Limit?" In: *Journal of the American Chemical Society* 134.36 (Aug. 2012), pp. 15016–15021. DOI: [10.1021/ja3055639](https://doi.org/10.1021/ja3055639).
- [26] Geoffrey E. Hinton et al. *Improving neural networks by preventing co-adaptation of feature detectors*. 2012. DOI: [10.48550/ARXIV.1207.0580](https://doi.org/10.48550/ARXIV.1207.0580).
- [27] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012.
- [28] Michael Fernandez et al. "Large-Scale Quantitative Structure-Property Relationship (QSPR) Analysis of Methane Storage in Metal-Organic Frameworks". In: *The Journal of Physical Chemistry C* 117.15 (2013), pp. 7681–7689. DOI: [10.1021/jp4006422](https://doi.org/10.1021/jp4006422).

- [29] Yongchul G. Chung, Jeffrey Camp, et al. "Computation-Ready, Experimental Metal-Organic Frameworks: A Tool To Enable High-Throughput Screening of Nanoporous Crystals". In: *Chemistry of Materials* 26.21 (2014), pp. 6185–6192. DOI: [10.1021/cm502594j](https://doi.org/10.1021/cm502594j).
- [30] G. James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer New York, 2014.
- [31] Richard L. Martin et al. "In Silico Design of Three-Dimensional Porous Covalent Organic Frameworks via Known Synthesis Routes and Commercially Available Species". In: *The Journal of Physical Chemistry C* 118.41 (Oct. 2014), pp. 23790–23802. DOI: [10.1021/jp507152j](https://doi.org/10.1021/jp507152j).
- [32] Nitish Srivastava et al. "Dropout: a simple way to prevent neural networks from overfitting". In: *J. Mach. Learn. Res.* 15.1 (Jan. 2014), pp. 1929–1958.
- [33] Kaiming He et al. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. 2015. DOI: [10.48550/ARXIV.1502.01852](https://doi.org/10.48550/ARXIV.1502.01852).
- [34] Sergey Ioffe and Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. 2015. DOI: [10.48550/ARXIV.1502.03167](https://doi.org/10.48550/ARXIV.1502.03167).
- [35] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *Nature* 521.7553 (May 2015), pp. 436–444. DOI: [10.1038/nature14539](https://doi.org/10.1038/nature14539).
- [36] Cory M. Simon, Jihan Kim, et al. "The materials genome in action: identifying the performance limits for methane storage". In: *Energy & Environmental Science* 8.4 (2015), pp. 1190–1199. DOI: [10.1039/c4ee03515a](https://doi.org/10.1039/c4ee03515a).
- [37] Cory M. Simon, Rocio Mercado, et al. "What Are the Best Materials To Separate a Xenon/Krypton Mixture?" In: *Chemistry of Materials* 27.12 (2015), pp. 4459–4475. DOI: [10.1021/acs.chemmater.5b01475](https://doi.org/10.1021/acs.chemmater.5b01475).
- [38] Debasis Banerjee et al. "Metal–organic framework with optimally selective xenon adsorption and separation". In: *Nature Communications* 7.1 (June 2016), ncomms11831. DOI: [10.1038/ncomms11831](https://doi.org/10.1038/ncomms11831).
- [39] Diego A. Gómez-Gualdrón et al. "Evaluating topologically diverse metal-organic frameworks for cryo-adsorbed hydrogen storage". In: *Energy Environ. Sci.* 9 (10 2016), pp. 3279–3289. DOI: [10.1039/C6EE02104B](https://doi.org/10.1039/C6EE02104B).
- [40] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [41] Charles R. Qi et al. *PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation*. 2016. DOI: [10.48550/ARXIV.1612.00593](https://doi.org/10.48550/ARXIV.1612.00593).
- [42] Ioannis Spanopoulos et al. "Reticular Synthesis of HKUST-like tbo-MOFs with Enhanced CH₄ Storage". In: *Journal of the American Chemical Society* 138.5 (Jan. 2016), pp. 1568–1574. DOI: [10.1021/jacs.5b11079](https://doi.org/10.1021/jacs.5b11079).
- [43] Pantelis N. Trikalitis et al. "Reticular Chemistry at Its Best: Directed Assembly of Hexagonal Building Units into the Awaited Metal-Organic Framework with the Intricate Polybenzene Topology, pbz-MOF". In: *Journal of the American Chemical Society* 138.39 (2016). PMID: 27615117, pp. 12767–12770. DOI: [10.1021/jacs.6b08176](https://doi.org/10.1021/jacs.6b08176).

- [44] Yamil J. Colón, Diego A. Gómez-Gualdrón, and Randall Q. Snurr. “Topologically Guided, Automated Construction of Metal–Organic Frameworks and Their Evaluation for Energy-Related Applications”. In: *Crystal Growth & Design* 17.11 (2017), pp. 5801–5810. DOI: [10.1021/acs.cgd.7b00848](https://doi.org/10.1021/acs.cgd.7b00848).
- [45] Aurélien Géron. *Hands-on machine learning with Scikit-Learn and TensorFlow: concepts, tools, and techniques to build intelligent systems*. Sebastopol, CA: O’Reilly Media, 2017.
- [46] WooSeok Jeong et al. “Modeling adsorption properties of structurally deformed metal–organic frameworks using structure–property map”. In: *Proceedings of the National Academy of Sciences* 114.30 (2017), pp. 7923–7928. DOI: [10.1073/pnas.1706330114](https://doi.org/10.1073/pnas.1706330114).
- [47] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017.
- [48] Peyman Z. Moghadam, Aurelia Li, et al. “Development of a Cambridge Structural Database Subset: A Collection of Metal–Organic Frameworks for Past, Present, and Future”. In: *Chemistry of Materials* 29.7 (2017), pp. 2618–2625. DOI: [10.1021/acs.chemmater.7b00441](https://doi.org/10.1021/acs.chemmater.7b00441).
- [49] Patrick Bleiziffer, Kay Schaller, and Sereina Riniker. “Machine Learning of Partial Charges Derived from High-Quality Quantum-Mechanical Calculations”. In: *Journal of Chemical Information and Modeling* 58.3 (Feb. 2018), pp. 579–590. DOI: [10.1021/acs.jcim.7b00663](https://doi.org/10.1021/acs.jcim.7b00663).
- [50] David Dubbeldam, Sofía Calero, and Thijs J.H. Vlugt. “iRASP: GPU-accelerated visualization software for materials scientists”. In: *Molecular Simulation* 44.8 (Jan. 2018), pp. 653–676. DOI: [10.1080/08927022.2018.1426855](https://doi.org/10.1080/08927022.2018.1426855).
- [51] Kyungdoc Kim et al. “Deep-learning-based inverse design model for intelligent discovery of organic molecules”. In: *npj Computational Materials* 4.1 (Dec. 2018). DOI: [10.1038/s41524-018-0128-1](https://doi.org/10.1038/s41524-018-0128-1).
- [52] Haichen Li et al. “Tuning the molecular weight distribution from atom transfer radical polymerization using deep reinforcement learning”. In: *Mol. Syst. Des. Eng.* 3 (3 2018), pp. 496–508. DOI: [10.1039/C7ME00131B](https://doi.org/10.1039/C7ME00131B).
- [53] Rocio Mercado et al. “In Silico Design of 2D and 3D Covalent Organic Frameworks for Methane Storage Applications”. In: *Chemistry of Materials* 30.15 (June 2018), pp. 5069–5086. DOI: [10.1021/acs.chemmater.8b01425](https://doi.org/10.1021/acs.chemmater.8b01425).
- [54] Peyman Z. Moghadam, Timur Islamoglu, et al. “Computer-aided discovery of a metal-organic framework with superior oxygen uptake”. In: *Nature Communications* 9.1 (Apr. 2018). DOI: [10.1038/s41467-018-03892-8](https://doi.org/10.1038/s41467-018-03892-8).
- [55] Michael A. Nielsen. *Neural Networks and Deep Learning*. misc. 2018.
- [56] Minman Tong et al. “Computation-Ready, Experimental Covalent Organic Framework for Methane Delivery: Screening and Material Design”. In: *The Journal of Physical Chemistry C* 122.24 (May 2018), pp. 13009–13016. DOI: [10.1021/acs.jpcc.8b04742](https://doi.org/10.1021/acs.jpcc.8b04742).
- [57] Tian Xie and Jeffrey C. Grossman. “Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties”. In: *Physical Review Letters* 120.14 (Apr. 2018). DOI: [10.1103/physrevlett.120.145301](https://doi.org/10.1103/physrevlett.120.145301).

- [58] Peter G. Boyd et al. "Data-driven design of metal-organic frameworks for wet flue gas CO₂ capture". In: *Nature* 576.7786 (Dec. 2019), pp. 253–256. DOI: [10.1038/s41586-019-1798-7](https://doi.org/10.1038/s41586-019-1798-7).
- [59] D.P. Broom et al. "Concepts for improving hydrogen storage in nanoporous materials". In: *International Journal of Hydrogen Energy* 44.15 (2019). A special issue on hydrogen-based Energy storage, pp. 7768–7779. DOI: <https://doi.org/10.1016/j.ijhydene.2019.01.224>.
- [60] Benjamin J. Bucior, N. Scott Bobbitt, et al. "Energy-based descriptors to rapidly predict hydrogen storage in metal-organic frameworks". In: *Mol. Syst. Des. Eng.* 4 (2019), pp. 162–174. DOI: [10.1039/C8ME00050F](https://doi.org/10.1039/C8ME00050F).
- [61] Benjamin J. Bucior, Andrew S. Rosen, et al. "Identification Schemes for Metal-Organic Frameworks To Enable Rapid Search and Cheminformatics Analysis". In: *Crystal Growth & Design* 19.11 (Sept. 2019), pp. 6682–6697. DOI: [10.1021/acs.cgd.9b01050](https://doi.org/10.1021/acs.cgd.9b01050).
- [62] Chi Chen et al. "Graph Networks as a Universal Machine Learning Framework for Molecules and Crystals". In: *Chemistry of Materials* 31.9 (Apr. 2019), pp. 3564–3572. DOI: [10.1021/acs.chemmater.9b01294](https://doi.org/10.1021/acs.chemmater.9b01294).
- [63] Yongchul G. Chung, Emmanuel Haldoupis, et al. "Advances, Updates, and Analytics for the Computation-Ready, Experimental Metal-Organic Framework Database: CoRE MOF 2019". In: *Journal of Chemical & Engineering Data* 64.12 (Nov. 2019), pp. 5985–5998. DOI: [10.1021/acs.jced.9b00835](https://doi.org/10.1021/acs.jced.9b00835).
- [64] Hana Dureckova et al. "Robust Machine Learning Models for Predicting High CO₂ Working Capacity and CO₂ Selectivity of Gas Adsorption in Metal-Organic Frameworks for Precombustion Carbon Capture". In: *The Journal of Physical Chemistry C* 123.7 (Jan. 2019), pp. 4133–4139. DOI: [10.1021/acs.jpcc.8b10644](https://doi.org/10.1021/acs.jpcc.8b10644).
- [65] George S. Fanourgakis, Konstantinos Gkagkas, Emmanuel Tylianakis, Emmanuel Klontzas, et al. "A Robust Machine Learning Algorithm for the Prediction of Methane Adsorption in Nanoporous Materials". In: *The Journal of Physical Chemistry A* 123.28 (2019), pp. 6080–6087. DOI: [10.1021/acs.jpca.9b03290](https://doi.org/10.1021/acs.jpca.9b03290).
- [66] Preetum Nakkiran et al. *Deep Double Descent: Where Bigger Models and More Data Hurt*. 2019. DOI: [10.48550/ARXIV.1912.02292](https://arxiv.org/abs/10.48550/ARXIV.1912.02292).
- [67] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems* 32. Curran Associates, Inc., 2019, pp. 8024–8035.
- [68] Omar M. Yaghi. "Emergence of Metal-Organic Frameworks". In: *Introduction to Reticular Chemistry*. John Wiley & Sons, Ltd, 2019. Chap. 1, pp. 1–27. DOI: <https://doi.org/10.1002/9783527821099.ch1>.
- [69] Fuzhen Zhuang et al. *A Comprehensive Survey on Transfer Learning*. 2019. DOI: [10.48550/ARXIV.1911.02685](https://arxiv.org/abs/10.48550/ARXIV.1911.02685).
- [70] Saifullahi Aminu Bello, Shangshu Yu, and Cheng Wang. *Review: deep learning on 3D point clouds*. 2020. DOI: [10.48550/ARXIV.2001.06280](https://arxiv.org/abs/10.48550/ARXIV.2001.06280).

- [71] George S. Fanourgakis, Konstantinos Gkagkas, Emmanuel Tylianakis, and George Froudakis. "A Generic Machine Learning Algorithm for the Prediction of Gas Adsorption in Nanoporous Materials". In: *The Journal of Physical Chemistry C* 124.13 (2020), pp. 7117–7126. DOI: [10 . 1021/acs . jpcc . 9b10766](https://doi.org/10.1021/acs.jpcc.9b10766).
- [72] Ruimin Ma, Yamil J. Colón, and Tengfei Luo. "Transfer Learning Study of Gas Adsorption in Metal-Organic Frameworks". In: *ACS Applied Materials & Interfaces* 12.30 (July 2020), pp. 34041–34048. DOI: [10 . 1021/acsami . 0c06858](https://doi.org/10.1021/acsami.0c06858).
- [73] Ali Raza et al. "Message Passing Neural Networks for Partial Charge Assignment to Metal-Organic Frameworks". In: *The Journal of Physical Chemistry C* 124.35 (Aug. 2020), pp. 19070–19082. DOI: [10 . 1021/acs . jpcc . 0c04903](https://doi.org/10.1021/acs.jpcc.0c04903).
- [74] Ying Wu, Haipeng Duan, and Hongxia Xi. "Machine Learning-Driven Insights into Defects of Zirconium Metal-Organic Frameworks for Enhanced Ethane-Ethylene Separation". In: *Chemistry of Materials* 32.7 (Mar. 2020), pp. 2986–2997. DOI: [10 . 1021 / acs . chemmater . 9b05322](https://doi.org/10.1021/acs.chemmater.9b05322).
- [75] Omar M. Yaghi. "The Reticular Chemist". In: *Nano Letters* 20.12 (Nov. 2020), pp. 8432–8434. DOI: [10 . 1021/acs . nanolett . 0c04327](https://doi.org/10.1021/acs.nanolett.0c04327).
- [76] Srinivasu Kanchalapalli et al. "Fast and Accurate Machine Learning Strategy for Calculating Partial Atomic Charges in Metal-Organic Frameworks". In: *Journal of Chemical Theory and Computation* 17.5 (Mar. 2021), pp. 3052–3064. DOI: [10 . 1021/acs . jctc . 0c01229](https://doi.org/10.1021/acs.jctc.0c01229).
- [77] Sangwon Lee et al. "Computational Screening of Trillions of Metal-Organic Frameworks for High-Performance Methane Storage". In: *ACS Applied Materials & Interfaces* 13.20 (2021), pp. 23647–23654. DOI: [10 . 1021/acsami . 1c02471](https://doi.org/10.1021/acsami.1c02471).
- [78] Omid T. Qazvini, Ravichandar Babarao, and Shane G. Telfer. "Selective capture of carbon dioxide from hydrocarbons using a metal-organic framework". In: *Nature Communications* 12.1 (Jan. 2021). DOI: [10 . 1038/s41467-020-20489-2](https://doi.org/10.1038/s41467-020-20489-2).
- [79] Andrew S. Rosen et al. "Machine learning the quantum-chemical properties of metal-organic frameworks for accelerated materials discovery". In: *Matter* 4.5 (May 2021), pp. 1578–1597. DOI: [10 . 1016/j . matt . 2021 . 02 . 015](https://doi.org/10.1016/j.matt.2021.02.015).
- [80] Kuthuru Suresh et al. "Optimizing Hydrogen Storage in MOFs through Engineering of Crystal Morphology and Control of Crystal Size". In: *Journal of the American Chemical Society* 143.28 (July 2021), pp. 10727–10734. DOI: [10 . 1021/jacs . 1c04926](https://doi.org/10.1021/jacs.1c04926).
- [81] Zhenpeng Yao et al. "Inverse design of nanoporous crystalline reticular materials with deep generative models". In: *Nature Machine Intelligence* 3.1 (Jan. 2021), pp. 76–86. DOI: [10 . 1038 / s42256-020-00271-1](https://doi.org/10.1038/s42256-020-00271-1).
- [82] Niklas W. A. Gebauer et al. "Inverse design of 3d molecular structures with conditional generative neural networks". In: *Nature Communications* 13.1 (Feb. 2022). DOI: [10 . 1038/s41467-022-28526-y](https://doi.org/10.1038/s41467-022-28526-y).
- [83] Stephen Gow et al. "A review of reinforcement learning in chemistry". In: *Digital Discovery* 1.5 (2022), pp. 551–567. DOI: [10 . 1039/d2dd00047d](https://doi.org/10.1039/d2dd00047d).

- [84] Chuanhai Jiang et al. "Recent advances in metal-organic frameworks for gas adsorption/separation". In: *Nanoscale Advances* 4.9 (2022), pp. 2077–2089. DOI: [10.1039/d2na00061j](https://doi.org/10.1039/d2na00061j).
- [85] Saptasree Bose et al. "Challenges and Opportunities: Metal-Organic Frameworks for Direct Air Capture". In: *Advanced Functional Materials* (2023). DOI: [10.1002/adfm.202307478](https://doi.org/10.1002/adfm.202307478).
- [86] Zhonglin Cao et al. "MOFormer: Self-Supervised Transformer Model for Metal-Organic Framework Property Prediction". In: *Journal of the American Chemical Society* 145.5 (Jan. 2023), pp. 2958–2967. DOI: [10.1021/jacs.2c11420](https://doi.org/10.1021/jacs.2c11420).
- [87] Juul S. De Vos et al. "ReDD-COFFEE: a ready-to-use database of covalent organic framework structures and accurate force fields to enable high-throughput screenings". In: *Journal of Materials Chemistry A* 11.14 (2023), pp. 7468–7487. DOI: [10.1039/d3ta00470h](https://doi.org/10.1039/d3ta00470h).
- [88] Yeonghun Kang et al. "A multi-modal pre-training transformer for universal transfer learning in metal-organic frameworks". In: *Nature Machine Intelligence* 5.3 (Mar. 2023), pp. 309–318. DOI: [10.1038/s42256-023-00628-2](https://doi.org/10.1038/s42256-023-00628-2).
- [89] Ibrahim B. Orhan et al. "Accelerating the prediction of CO₂ capture at low partial pressures in metal-organic frameworks using new machine learning descriptors". In: *Communications Chemistry* 6.1 (Oct. 2023). DOI: [10.1038/s42004-023-01009-x](https://doi.org/10.1038/s42004-023-01009-x).
- [90] Kaihang Shi et al. "Two-Dimensional Energy Histograms as Features for Machine Learning to Predict Adsorption in Diverse Nanoporous Materials". In: *Journal of Chemical Theory and Computation* (Feb. 2023). DOI: [10.1021/acs.jctc.2c00798](https://doi.org/10.1021/acs.jctc.2c00798).
- [91] Constantinos Tsangarakis et al. "Water-Stable etb-MOFs for Methane and Carbon Dioxide Storage". In: *Inorganic Chemistry* 62.14 (2023), pp. 5496–5504. DOI: [10.1021/acs.inorgchem.2c04483](https://doi.org/10.1021/acs.inorgchem.2c04483).

Index

Symbols

LeakyReLU 25, 43
ReLU 25

A

Ab-initio calculations 42
Activation function 25
Adam 39, 40
Adaptive learning rate 39
Agent 16
AlexNet 15
Architecture 23, 26, 30

B

Back-propagation 36, 37
Band gap 51
Batch gradient descent 39
Batch normalization 35
Batch normalization layer 35
Batch size 38
Bias 19, 21, 24
Bias trick 24
Bias-variance decomposition 21
Bias-variance trade-off 22
Big data 12, 15
Binary classifier 24
Binary cross entropy loss 18
Boltzmann constant 43
Bulk modulus 51

C

Calculus 36
Capacity 20
Carbon capture 10
Catalysis 10
Chain rule 36
Chemical system 51
Chemistry 16, 51
Classification 16, 27
Classification accuracy 22
CNN training 44

Coefficient of determination 43
Complexity 20, 21
Complexity curve 23
Composite function 27, 36
Computational cost 14, 42
Computational graph 25
Computational screening 11
Conditional generative modelling 16
Confidence interval 45, 49
Convergence 39
Convolutional filter 32
Convolutional layer 32, 43, 46
Convolutional neural network 15
Coordinates 51
CoRE MOF database 13
Covalent organic frameworks 11
Crystal graph 51
Cutoff radius 42

D

Data 12, 15
Data augmentation 29, 44, 45, 48
Data efficiency 51
Data generating distribution 28
Data-driven 12
Database 11
Dataset 15, 21, 41
Deep learning 15, 23
Deep learning algorithm 51
Dense layer 33, 46
Derivative 25
Descriptor 12, 16, 51
Dipole moment 49
Direct air capture 10
Directed graph 26
Distribution 23, 26
Double descent 22
Downsample 46
Dropout 29, 43
Dropout rate 30, 43
Drug delivery 10

E		
Effective capacity	20	
Electron density	51	
Electrostatic interactions	49	
Empirical risk minimization	18	
Energetic fingerprint	13, 47	
Energy grid	13	
Energy histogram	13	
Energy image	14	
Energy voxels	43	
Energy-based descriptors	13, 46	
Epoch	44	
Expected square loss	21	
Experience	15, 21, 23	
Experimental characterization	11	
Experimental synthesis	11	
Exploding gradients	34	
Extracted feature	27	
F		
Feature	14, 16, 22	
Feature extraction	47	
Feature map	31, 46, 47	
Feedforward network	28	
Filter	30	
Fine-tune	51	
Fingerprint extraction	47	
Flatten layer	46	
Forward pass	36, 47	
Fully connected neural network	26	
Function	24	
G		
Gas	46	
Gas adsorption	13, 46	
Gas separation	10	
Gas storage	10	
Gas uptake	13, 41	
Generalization ability	43	
Generalization error	17	
Generalization loss	17, 20, 28	
Generative modeling	16	
Geometric descriptors	13, 46	
Geometric transformations	29, 44	
Gradient descent	20	
Gradient operator	37	
Gradient-based optimization	25	
Graph	25	
Gravimetric surface area	13	
Grayscale image	43	
Guest molecule	13	
H		
Hand-crafted fingerprints	47	
Heavyside step function	25	
Hidden layer	26	
Hidden unit	26	
High complexity	23	
High level representation	47	
High-level feature	27	
hMOFs database	13	
Horizontal edge	33	
Host-guest interactions	13, 51	
Hydrogen storage	10	
Hydrogen storage.	11	
Hyperbolic tangent	25, 35	
Hyperparameter	19, 39	
Hypothesis	17	
Hypothesis space	18	
Hypothetical MOFs	41	
I		
Image classification	15, 27	
Image-like data	30, 48	
ImageNet	15	
Index phase	27	
Inductive bias	29	
Information content	42	
Inner product	31	
Input	12, 16	
Input layer	26, 47	
Intelligent behavior	23, 24	
Intelligent system	24	
Inverse design	16	
Inverted dropout	30	
IRMOF-1	11, 47	
Irreducible error	21	
K		
Keep probability	30	
Kernel	30	
Kernel size	33, 43	
L		
Label	16, 29, 41, 44	
Language translation	15	
Lasso regression	13	
Lasso regularization	19	
Learner	22, 28	



Learning algorithm 18
 Learning curve 22, 41, 42, 48
 Learning paradigms 16
 Learning rate 20, 38
 Lennard-Jones potential 42
 Linear binary classifier 24
 Linear function 24
 Linear layer 47
 Linear model 27
 Linear regression 18, 29
 Linearly separable 28
 Local minimum 39
 Lorentz-Berthelot mixing rules 43
 Loss function 17
 Low complexity 21, 23

M

Machine learning 12, 15
 Machine learning algorithm 12
 Macroscopic objects 24
 Mapping 27
 Material 12
 Material space 12
 MaxPool layer 43
 Mean absolute error 13
 Mean squared loss 19
 Metal clusters 10
 Metal ions 10
 Metal-organic frameworks 10
 Methane storage 10
 Mini-batch gradient descent 39
 Model 12, 18
 MOF-5 11
 Molecular simulations 11, 41
 Molecule 12
 Momentum 39
 Multi-label classification 16
 Multi-label regression 16
 Multi-valued output 27
 Multilayer perceptron 26

N

Natural language processing 28
 Neural network 41
 Neuron 23, 24, 27
 Neuroscience 23
 Non-convex optimization 12
 Non-linearity 24

O

Object detection 30
 Object recognition 32
 Occam's razor 19
 Optimistically biased 21
 Optimization 12, 37
 Optimization problem 12
 Optimizer 39
 Organic ligand 10
 Organic linker 10
 Output 12, 16, 27
 Output layer 26, 27, 46
 Overfitting 44

P

Padding 43
 Parallelization 43
 Parameter sharing 32
 Parity plot 49, 50
 Partial charge 49
 Perceptron 24
 Performance 20, 21, 23
 Performance measure 15
 performance metric 22
 Piecewise linear function 25
 Point cloud 51
 Polynomial regression 19
 Pooling layer 46
 Pore limiting diameter 13
 Potential energy surface 14
 Pre-trained model 51
 Predictor 16
 Pressure 13
 Principle of parsimony 19
 Probe molecule 42
 Probe particle 13
 PyTorch 43

Q

Quadrupole moment 49

R

Random error term 17
 Random forest 13
 Random guessing 28
 Reflection 44
 Regression 13, 16, 27
 Regularization 19
 Regularization techniques 29



Regularizer 19
 Reinforcement learning 16
 Representation 51
 Representational capacity 18, 20
 Reticular chemistry 10, 23, 51
 Reticular materials 10, 51
 RetNet 43, 46, 47
 Rotation 44

S

Sample 43
 Scikit-learn 43
 Semantic segmentation 30
 Sigmoid 25, 35
 Source task 51
 Spam filter 15
 Sparse connections 32
 Spatio-temporal relationship 30
 Squared loss 21
 Squared loss 18
 Stochastic gradient descent 39
 Stride 43
 Subatomic particles 24
 Supervised learning 12, 16
 Surface area 11
 Swing capacity 13
 Synapses 27

T

Target task 51
 Task 15, 22
 Temperature 43
 Test accuracy 23
 Test error 21
 Test loss 21
 Test performance 22
 Test set 21, 41, 43, 45
 Text representation 51
 Textual properties 41

Thermodynamic conditions 13, 41, 51
 Top-5 accuracy 15
 Traditional programming 16
 Training accuracy 22
 Training data 22, 33, 36, 41, 51
 Training error 18
 Training instance 16
 Training loss 18, 20, 28, 29, 36, 37
 Training performance 22
 Training phase 16, 19, 21, 47
 Training sample 16
 Training set 17, 21, 22, 29, 38, 41, 44
 Transfer learning 51

U

Unbiased estimate 21
 Unit cell 13
 Universal approximation theorem 28
 Unstable gradients 35
 Unsupervised learning 16
 UO database 41

V

Validation set 21, 41, 44
 Van der Waals interactions 13
 Vanishing gradients 34
 Vapnik-Chervonenkis dimension 18
 Variance 21, 22
 Void fraction 13
 Voxelization 42
 Voxelized electron density 51
 Voxelized PES 42, 46

W

Weighted sum 24
 Weights 19, 44

X

XOR problem 27

Acronyms

LeakyReLU leaky rectified linear unit.

ReLU rectified linear unit.

ANG adsorbed natural gas.

BGD batch gradient descent.

CI confidence interval.

CNG compressed natural gas.

CNN convolutional neural network.

COF covalent organic framework.

CoRE Computation-Ready Experimental.

CSD Cambridge Structural Database.

DAC direct air capture.

DAG directed acyclic graph.

DL deep learning.

FCNN fully connected neural network.

GCMC Grand Canonical Monte Carlo.

GPU graphics processing unit.

i.i.d independent and identically distributed.

ILSVRC Large Scale Visual Recognition Challenge.

LJ Lennard-Jones.

LNG liquefied natural gas.

MAE mean absolute error.

MBGD mini-batch gradient descent.

ML machine learning.

MLP multilayer perceptron.

MOF metal-organic framework.

MSL mean squared loss.

NaN Not a Number.

NLP natural language processing.

NN neural network.

PES potential energy surface.

RF Random Forest.

SGD stochastic gradient descent.

SVM support vector machine.

UO University of Ottawa.

Appendix A

MOX $\epsilon\lambda$

```
# This file is part of MOX .
# Copyright (C) 2023-2024 Antonios P. Sarikas

# MOX is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.

# This program is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
# GNU General Public License for more details.

# You should have received a copy of the GNU General Public License
# along with this program. If not, see <http://www.gnu.org/licenses/>.

r"""
This module provides helper functions for creating voxels.

.. note::
    Currently, interactions are modelled with the Lennard-Jones (LJ)
    ↪ potential.

.. attention::
    Consider tuning the ``n_jobs`` parameter to get the best
    ↪ performance
    for your system::

        from timeit import timeit

        setup = 'from moxel.utils import voxels_from_file'
        n_jobs = [1, 2, 8, 16] # Modify this according to your system.
```

```

        for n in n_jobs:
            stmt = f'voxels_from_file("path/to/cif", n_jobs={n})'
            time = timeit(stmt=stmt, setup=setup, number=1)
            print(f'Time with {n} jobs: {time:.3f} s')

"""

import os
import itertools
from pathlib import Path
from multiprocessing import Pool
import warnings

import numpy as np
from tqdm import tqdm
from pymatgen.core import Structure

from ._params import lj_params

warnings.filterwarnings('ignore')

# Default values for voxels calculation.
GRID_SIZE = 32
CUTOFF = 10.
EPSILON = 50.
SIGMA = 2.5
CUBIC_BOX = 30.
N_JOBS = None

def mic_scale_factors(r, lattice_vectors):
    """
    Return scale factors to satisfy minimum image convention [MIC]_.

    Parameters
    -----
    r : float
        The cutoff radius used in MIC convention.
    lattice_vectors : array of shape (3, 3)
        The lattice vectors of the unit cell.
        Each row corresponds to a lattice vector.

    Returns
    -----
    """

```

```

scale_factors : array of shape (3,)
    ``scale_factors[i]`` scales ``lattice_vectors[i]``.

References
-----
.. [MIC] W. Smith, "The Minimum Image Convention in Non-Cubic MD
   ↪ Cells", 1989.
"""
a, b, c = lattice_vectors
volume = np.linalg.norm(np.dot(a, np.cross(b, c)))

w_a = volume/np.linalg.norm(np.cross(b, c))
w_b = volume/np.linalg.norm(np.cross(a, c))
w_c = volume/np.linalg.norm(np.cross(a, b))

return np.ceil(2 * r / np.array([w_a, w_b, w_c]))

class Grid:
    r"""
    A 3D energy grid over a crystal structure.

    Parameters
    -----
    grid_size : int, default=32
        Number of grid points along each dimension.
    cutoff : float, default=10.0
        Cutoff radius (Å) for the LJ potential.
    epsilon : float, default=50.0
        Epsilon value (/K) of the probe atom.
    sigma : float, default=2.5
        Sigma value (/Å) of the probe atom.

    Attributes
    -----
    structure : :class:`pymatgen.core.structure.Structure`
        Available only after :meth:`Grid.load_structure` has been
        ↪ called.
    structure_name : str
        Available only after :meth:`Grid.load_structure` has been
        ↪ called.
    cubic_box : float or None
        Available only after :meth:`Grid.calculate` has been called.

```

```

voxels : array of shape (grid_size,)*3
    Available only after :meth:`Grid.calculate` has been called.
"""
def __init__(
    self,
    grid_size=GRID_SIZE,
    *,
    cutoff=CUTOFF,
    epsilon=EPSILON,
    sigma=SIGMA
):
    self.grid_size = grid_size
    self.cutoff = cutoff
    self.epsilon = epsilon
    self.sigma = sigma

def load_structure(self, pathname):
    r"""
    Load a crystal structure from a file in a format supported by
    :meth:`pymatgen.core.Structure.from_file`.

    Parameters
    -----
    pathname : str
        Pathname to the file.
    """
    self.structure = Structure.from_file(pathname)
    self.structure_name = Path(pathname).stem

def calculate(
    self,
    cubic_box=CUBIC_BOX,
    potential='lj',
    n_jobs=N_JOBS,
):
    r"""
    Iterate over the grid and return voxels.

    For computational efficiency and to assure (approximately) the
    ↪ same
    spatial resolution, the grid is overlayed over a supercell
    ↪ scaled
    according to MIC, see :func:`mic_scale_factors`.

```

*If lattice angles are significantly different than 90°, to
 → avoid
 distortions set ``cubic\_box``. In this case, the grid is
 → overlayed over
 a cubic box of size ``cubic\_box`` centered at the origin but
 → periodicity
 is no longer guaranteed.*

Parameters

potential : str, default='lj'
*The potential used to calculate voxels. Currently, only the
 LJ potential is supported.*

cubic_box : float or None, default=30.0
*If ``None``, the simulation box is a supercell scaled
 → according to
 MIC. Otherwise, cubic box of size ``cubic\_box``.*

n_jobs : int, optional
*Number of jobs to run in parallel. If ``None``, then the
 → number returned
 by ``os.cpu\_count()`` is used.*

Returns

*voxels : array of shape (grid_size,)*3*
 """

```

self.cubic_box = cubic_box

if cubic_box is not None:
    d = cubic_box / 2
    probe_coords = np.linspace(0 - d, 0 + d, self.grid_size,
    → endpoint=False) # Cartesian
    self._simulation_box = self.structure
else:
    probe_coords = np.linspace(0, 1, self.grid_size,
    → endpoint=False) # Fractional
    scale = mic_scale_factors(self.cutoff,
    → self.structure.lattice.matrix)
    self._simulation_box = self.structure * scale

if potential == 'lj':
    # Cache LJ parameters for all atoms in the simulation box.
  
```

```

        self._lj_params = np.array(
            [lj_params[atom.species_string] for atom in
             ↪ self._simulation_box]
        )

        # Cache fractional coordinates since this is a slow
        ↪ function in pymatgen.
        self._frac_coors = self._simulation_box.frac_coors

        # Embarrassingly parallel.
        with Pool(processes=n_jobs) as p:
            energies = p.map(
                self.lj_potential,
                ↪ itertools.product(*(probe_coors,)*3)
            )

        self.voxels = np.array(energies,
            ↪ dtype=np.float32).reshape((self.grid_size,)*3)

        return self.voxels

def lj_potential(self, coords):
    r"""
    Calculate LJ potential at cartesian or fractional
    coordinates.

    Parameters
    -----
    coords : array_like of shape (3,)
        If ``cubic_box=None`` fractional, else cartesian.

    Returns
    -----
    energy : float
    """
    if self.cubic_box is not None:
        cartesian_coors = coords
    else:
        cartesian_coors =
            ↪ self._simulation_box._lattice.get_cartesian_coors(coords)

    _, r_ij, indices, _ =
        ↪ self._simulation_box._lattice.get_points_in_sphere(

```

```

        self._frac_coords, cartesian_coords,
        self.cutoff, zip_results=False,
    )

    # Need to check for length of r_ij because of
    # https://github.com/materialsproject/pymatgen/issues/3794

    if len(r_ij) == 0: # No neighbor, zero energy.
        return 0.

    if np.any(r_ij < 1e-3): # Close contact, infinite energy.
        return np.inf

    es_j = self._lj_params[indices]
    x = (0.5 * (es_j[:, 1] + self.sigma)) / r_ij
    e = 4 * np.sqrt(es_j[:, 0] * self.epsilon)

    return np.sum(e * (x**12 - x**6))

def voxels_from_file(
    cif_pathname,
    grid_size=GRID_SIZE,
    *,
    cutoff=CUTOFF,
    epsilon=EPSILON,
    sigma=SIGMA,
    cubic_box=CUBIC_BOX,
    n_jobs=N_JOBS,
    only_voxels=True,
):
    r"""
    Calculate and return voxels from ``.cif`` file.

    Parameters
    -----
    cif_pathname : str
        Pathname to the ``.cif`` file.
    only_voxels : bool, default=True
        Determines ``out`` type.

    Returns
    -----

```

```

out : array or :class:`Grid`
    If ``only_voxels=True`` array, else :class:`Grid`.

See Also
-----

:func:`voxels_from_dir`
    For a description of the parameters.
"""
grid = Grid(grid_size, cutoff=cutoff, epsilon=epsilon, sigma=sigma)

grid.load_structure(cif_pathname)
grid.calculate(cubic_box=cubic_box, n_jobs=n_jobs)

if only_voxels:
    return grid.voxels

return grid

def voxels_from_files(
    cif_pathnames,
    out_pathname,
    grid_size=GRID_SIZE,
    *,
    cutoff=CUTOFF,
    epsilon=EPSILON,
    sigma=SIGMA,
    cubic_box=CUBIC_BOX,
    n_jobs=N_JOBS,
):
    r"""
    Calculate voxels from a list of ``.cif`` files and store them.

    Voxels are stored under ``out_pathname`` as ``.npy`` files.

    Parameters
    -----
    cif_pathnames : list
        List of pathnames to the ``.cif`` files.
    out_pathname : str
        Pathname to the directory under which voxels are stored.

    See Also

```



```

-----
:func: `voxels_from_dir`
    For a description of the parameters.

Notes
-----
Structures that can't be processed are omitted.
"""
os.mkdir(out_pathname)

for file in tqdm(cif_pathnames, desc='\033[32;1mCreating energy
↪ voxels\033[0m'):
    try:
        name = Path(file).stem # Name of the structure.
        grid = voxels_from_file(
            cif_pathname=file,
            grid_size=grid_size,
            cutoff=cutoff,
            epsilon=epsilon,
            sigma=sigma,
            cubic_box=cubic_box,
            n_jobs=n_jobs,
        )

        pathname = os.path.join(out_pathname, name)
        np.save(pathname, grid) # .npy extension is appended by
↪ default.
    except Exception as e:
        print(e)

def voxels_from_dir(
    cif_dirname: str,
    out_pathname: str,
    grid_size: int = GRID_SIZE,
    *,
    cutoff: float = CUTOFF,
    epsilon: float = EPSILON,
    sigma: float = SIGMA,
    cubic_box: float | None = CUBIC_BOX,
    n_jobs: int | None = N_JOBS,
):
    r"""

```

Calculate voxels from a directory of ``.cif`` files and store them.

Voxels are stored under ``out\_pathname`` as ``.npy`` files.

Parameters

`cif_dirname` : str

Pathname to the directory containing the ``.cif`` files.

`out_pathname` : str

Pathname of an existing directory under which voxels are
→ stored.

`grid_size` : int, default=32

Number of grid points along each dimension.

`cutoff` : float, default=10.0

Cutoff radius (Å) for the LJ potential.

`epsilon` : float, default=50.0

Epsilon value (/K) of the probe atom.

`sigma` : float, default=2.5

Sigma value (/Å) of the probe atom.

`cubic_box` : float or None, default=30

If ``None``, the simulation box is a supercell scaled according
→ to

MIC. Otherwise, cubic box of size ``cubic\_box``.

`n_jobs` : int, optional

Number of jobs to run in parallel. If ``None``, then the number
→ returned

by ``os.cpu\_count()`` is used.

Notes

Structures that can't be processsed are omitted.

"""

```
cif_pathnames = [os.path.join(cif_dirname, f) for f in
```

```
→ os.listdir(cif_dirname)]
```

```
voxels_from_files(
```

```
    cif_pathnames, out_pathname,
```

```
    grid_size=grid_size,
```

```
    cutoff=cutoff,
```

```
    epsilon=epsilon,
```

```
    sigma=sigma,
```

```
    cubic_box=cubic_box,
```

```
    n_jobs=n_jobs,
```

)

Appendix B

RetNet

```
import json
import torch
import numpy as np
import pandas as pd
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset
from itertools import cycle, combinations
from torch.utils.tensorboard import SummaryWriter
from torcheval.metrics.functional import r2_score

class RetNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv3d(in_channels=1, out_channels=12, kernel_size=3,
                ↪ padding=1, padding_mode='circular', bias=False),
            nn.BatchNorm3d(num_features=12),
            nn.LeakyReLU(),
        )
        self.conv2 = nn.Sequential(
            nn.Conv3d(in_channels=12, out_channels=24, kernel_size=3,
                ↪ bias=False),
            nn.BatchNorm3d(num_features=24),
            nn.LeakyReLU(),
        )

        self.max1 = nn.MaxPool3d(kernel_size=2)

        self.conv3 = nn.Sequential(
            nn.Conv3d(in_channels=24, out_channels=32, kernel_size=2,
                ↪ bias=False),
```

```

        nn.BatchNorm3d(num_features=32),
        nn.LeakyReLU(),
    )

    self.max2 = nn.MaxPool3d(kernel_size=2)

    self.conv4 = nn.Sequential(
        nn.Conv3d(in_channels=32, out_channels=64, kernel_size=2,
            ↪ bias=False),
        nn.BatchNorm3d(num_features=64),
        nn.LeakyReLU(),
    )
    self.conv5 = nn.Sequential(
        nn.Conv3d(in_channels=64, out_channels=120, kernel_size=2,
            ↪ bias=False),
        nn.BatchNorm3d(num_features=120),
        nn.LeakyReLU(),
    )
    self.fc = nn.Sequential(
        nn.Flatten(1),
        nn.Dropout(0.3),
        nn.Linear(3*3*3*120, 84),
        nn.BatchNorm1d(num_features=84),
        nn.LeakyReLU(),
        nn.Linear(84, 20),
        nn.BatchNorm1d(num_features=20),
        nn.LeakyReLU(),
        nn.Linear(20, 1),
    )

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = self.max1(x)
        x = self.conv3(x)
        x = self.max2(x)
        x = self.conv4(x)
        x = self.conv5(x)
        x = self.fc(x)

    return x

```

```

class VoNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv3d(in_channels=1, out_channels=12, kernel_size=3,
                ↪ padding=1, padding_mode='circular', bias=False),
            nn.BatchNorm3d(num_features=12),
            nn.LeakyReLU(),
        )
        self.conv2 = nn.Sequential(
            nn.Conv3d(in_channels=12, out_channels=24, kernel_size=3,
                ↪ bias=False),
            nn.BatchNorm3d(num_features=24),
            nn.LeakyReLU(),
        )

        self.max1 = nn.MaxPool3d(kernel_size=2)

        self.conv3 = nn.Sequential(
            nn.Conv3d(in_channels=24, out_channels=32, kernel_size=2,
                ↪ bias=False),
            nn.BatchNorm3d(num_features=32),
            nn.LeakyReLU(),
        )

        self.max2 = nn.MaxPool3d(kernel_size=2)

        self.conv4 = nn.Sequential(
            nn.Conv3d(in_channels=32, out_channels=64, kernel_size=2,
                ↪ bias=False),
            nn.BatchNorm3d(num_features=64),
            nn.LeakyReLU(),
        )
        self.conv5 = nn.Sequential(
            nn.Conv3d(in_channels=64, out_channels=120, kernel_size=2,
                ↪ bias=False),
            nn.BatchNorm3d(num_features=120),
            nn.LeakyReLU(),
        )
        self.fc = nn.Sequential(
            nn.Flatten(1),
            nn.Dropout(0.3),
            nn.Linear(3*3*3*120, 84),

```

```

        nn.BatchNorm1d(num_features=84),
        nn.LeakyReLU(),
        nn.Linear(84, 20),
        nn.BatchNorm1d(num_features=20),
        nn.LeakyReLU(),
        nn.Linear(20, 1),
    )

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = self.max1(x)
        x = self.conv3(x)
        x = self.max2(x)
        x = self.conv4(x)
        x = self.conv5(x)
        x = self.fc(x)

        return x

class LearningMethod:
    def __init__(self, network, optimizer, criterion):
        self.net = network
        self.optimizer = optimizer
        self.criterion = criterion

    def train(
        self, train_loader, val_loader,
        val_loss_freq=15, epochs=1, scheduler=None,
        metric=r2_score, device=None, tb_writer=None, verbose=True,
    ):

        self.scheduler = scheduler
        self.val_loss_freq = val_loss_freq
        self.train_hist = []
        self.train_metric = []
        self.val_hist = []
        self.val_metric = []
        self.writer = tb_writer
        self.train_batch_size = train_loader.batch_size
        self.val_batch_size = val_loader.batch_size
        self.epochs = epochs

```

```

val_loader = cycle(val_loader)

# Training phase.
counter = 0
for e in range(epochs):

    if verbose:
        print(f'\nEpoch: {e}')

    for i, (X_train, y_train) in enumerate(train_loader):
        X_train, y_train = X_train.to(device),
        ↪ y_train.to(device)

        # Initialize zero gradients.
        self.optimizer.zero_grad()

        y_train_hat = self.net(X_train)
        train_loss = self.criterion(y_train_hat.ravel(),
        ↪ y_train)

        if (i % val_loss_freq == 0):
            counter += val_loss_freq

            X_val, y_val = next(val_loader)
            X_val, y_val = X_val.to(device), y_val.to(device)

            # Account for Dropout + BatchNorm.
            yth = self.predict(X_train)

            y_val_hat = self.predict(X_val)
            val_loss = self.criterion(y_val_hat.ravel(), y_val)

            train_metric = metric(yth.ravel(), y_train)
            val_metric = metric(y_val_hat.ravel(), y_val)

            # Update the parameters.
            train_loss.backward()
            self.optimizer.step()

            # Print train and validation loss per `val_loss_freq`
            ↪ is.
            if verbose and (i % val_loss_freq == 0):

```



```

        print(
            f'{"Iteration {counter}":<20} ->',
            #f'{"train_loss = {train_loss:.3f}":<22}',
            f'{"train_metric = {train_metric:.3f}":<22}',
            #f'{"val_loss = {val_loss:.3f}":>22}', sep=4* ' '
            f'{"val_metric = {val_metric:.3f}":>22}', sep=4* ' '
            ↵ '
        )

    self.train_hist.append(train_loss.item())
    self.train_metric.append(train_metric.item())
    self.val_hist.append(val_loss.item())
    self.val_metric.append(val_metric.item())

    if tb_writer:
        self.writer.add_scalars(
            'learning_curve',
            #{'train': train_loss, 'val': val_loss},
            {'train': train_metric, 'val': val_metric},
            e
        )
        self.writer.add_scalar('Metric/train', train_metric, e)
        self.writer.add_scalar('Metric/val', val_metric, e)

        for name, value in self.net.named_parameters():
            self.writer.add_histogram(f'Values/{name}', value,
            ↵ e)
            self.writer.add_histogram(f'Gradients/{name}',
            ↵ value.grad, e)

    if scheduler:
        self.scheduler.step()

    if tb_writer:
        self.writer.flush()
        self.writer.close()

    print('\nTraining finished!')

def predict(self, X):
    self.net.eval()
    with torch.no_grad():
        y_pred = self.net(X)

```

```

        self.net.train()

        return y_pred

class CustomDataset(Dataset):
    def __init__(self, X, y, transform_X=None, transform_y=None):
        self.transform_X = transform_X
        self.transform_y = transform_y
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        sample_x = torch.tensor(self.X[idx])
        sample_y = torch.tensor(self.y[idx])

        if self.transform_X:
            sample_x = self.transform_X(sample_x)
        if self.transform_y:
            sample_y = self.transform_y(sample_y)

        return sample_x, sample_y

class Rotate90:
    def __init__(self):
        self.planes = list(combinations([1, 2, 3], 2))
        self.n_choices = len(self.planes)

    def __call__(self, sample):
        plane = self.planes[np.random.choice(self.n_choices)]
        direction = np.random.choice([-1, 1])

        return torch.rot90(sample, k=direction, dims=plane)

class Flip:
    def __call__(self, sample):
        axis = np.random.choice([1, 2, 3])

```

```

        return torch.flip(sample, [axis])

class Reflect:
    def __init__(self):
        self.planes = list(combinations([1, 2, 3], 2))
        self.n_choices = len(self.planes)

    def __call__(self, sample):
        plane = self.planes[np.random.choice(self.n_choices)]

        return torch.transpose(sample, *plane)

class Roll:
    def __call__(self, sample):
        axis = np.random.choice([1, 2, 3])
        shift = np.random.choice([1, 2, 4, 6, 10])
        direction = np.random.choice([-1, 1])

        return torch.roll(sample, shifts=shift * direction, dims=axis)

class Identity:
    def __call__(self, sample):

        return sample

@torch.no_grad()
def init_weights(m, initialization='normal', **kwargs):
    if initialization == 'normal':
        if type(m) == nn.Linear:
            m.weight = nn.init.kaiming_normal_(m.weight, **kwargs)

    elif initialization == 'uniform':
        if type(m) == nn.Linear:
            m.weight = nn.init.kaiming_uniform_(m.weight, **kwargs)

def load_data(dir_batch, path_to_csv, target_name, index_name_csv,
    ↪ size=None):
    with open(f'{dir_batch}/clean_names.json', 'r') as fhand:

```

```
names = json.load(fhand)['names']

df = pd.read_csv(path_to_csv)
df.set_index(index_name_csv, inplace=True)

y = df.loc[names, target_name].values.astype('float32')
X = np.load(f'{dir_batch}/clean_voxels.npy', mmap_mode='c')

return X[:size], y[:size]
```

Appendix C

Training RetNet

```
import torch
import random
import numpy as np
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
from torch.utils.data import DataLoader
from torcheval.metrics.functional import r2_score
from model import CustomDataset, Flip, Roll, Rotate90, Reflect,
    ↪ Identity
from torch.utils.tensorboard import SummaryWriter
from model import load_data, LearningMethod, RetNet, init_weights

# For reproducible results.
# See also -> https://pytorch.org/docs/stable/notes/randomness.html
np.random.seed(1)
torch.manual_seed(1)
random.seed(1)

# Requires installation with GPU support.
# See also -> https://pytorch.org/get-started/locally/
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
train_size = 32_432

# Load training data.
X_train, y_train = load_data(
    'data/MOFs/batch_train',
    'data/MOFs/all_MOFs_screening_data.csv',
    'CO2_uptake_P0.15bar_T298K [mmol/g]',
    'MOFname',
    size=train_size
)
```

```

# Load validation data.
X_val, y_val = load_data(
    'data/MOFs/batch_val_test',
    'data/MOFs/all_MOFs_screening_data.csv',
    'CO2_uptake_P0.15bar_T298K [mmol/g]',
    'MOFname',
    size=5_000
)

# Transformations for standardization + data augmentation.
standardization = transforms.Normalize(X_train.mean(), X_train.std())

augmentation = transforms.Compose([
    standardization,
    transforms.RandomChoice([Rotate90(), Flip(), Reflect(),
    ↪ Identity()]),
])

# Adding a channel dimension required for CNN.
X_train, X_val = [X.reshape(X.shape[0], 1, *X.shape[1:]) for X in
    ↪ [X_train, X_val]]

# Create the dataloaders.
train_loader = DataLoader(
    CustomDataset(X=X_train, y=y_train, transform_X=augmentation),
    batch_size=64, shuffle=True, pin_memory=True,
)

val_loader = DataLoader(
    CustomDataset(X=X_val, y=y_val, transform_X=standardization),
    batch_size=512, pin_memory=True,
)

# Define the architecture, loss and optimizer.
net = RetNet().to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(net.parameters(), lr=1e-3)

# Define the learning rate scheduler.
scheduler = optim.lr_scheduler.StepLR(
    optimizer, step_size=10,
    gamma=0.5, verbose=True
)

```

```

# Initialize weights.
net.apply(lambda m: init_weights(m, a=0.01))

# Initialize bias of the last layer with  $E[y_{train}]$ .
torch.nn.init.constant_(net.fc[-1].bias, y_train.mean())

model = LearningMethod(net, optimizer, criterion)
print(net)
model_name = 'RetNet'

# Use Tensorboard.
# See also ->
↪ https://pytorch.org/tutorials/recipes/recipes/tensorboard\_with\_pytorch.html
writer = SummaryWriter(log_dir='experiments/')

model.train(
    train_loader=train_loader, val_loader=val_loader,
    metric=r2_score, epochs=1, scheduler=scheduler,
    device=device, verbose=True, tb_writer=writer,
)

# Calculate R2 on the whole validation set.
predictions = []
for x, _ in val_loader:
    y_pred = model.predict(x.to(device))
    predictions.append(y_pred)

y_pred = torch.concatenate([b for b in predictions])
y_true = torch.tensor(y_val).reshape(len(y_val), -1).to(device)

print(f'R2 for validation set: {r2_score(y_pred, y_true)}')

# Save the trained model.
# See also ->
↪ https://pytorch.org/tutorials/beginner/saving\_loading\_models.html
torch.save(model, f'{model_name}.pt')

```

Appendix D

Training Random Forest

```
import json
import joblib
import numpy as np
import pandas as pd
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error as mae
from sklearn.metrics import median_absolute_error as mae
from sklearn.metrics import mean_absolute_percentage_error as mape
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

# For reproducible results.
np.random.seed(1)

# Load the names of materials used as train data.
with open('data/MOFs/batch_train/clean_names.json', 'r') as fhand:
    mof_train = json.load(fhand)['names']

# Load the names of materials used as test data.
with open('data/MOFs/batch_val_test/clean_names.json', 'r') as fhand:
    mof_test = json.load(fhand)['names'][5000:]

column_names = [
    'MOFname', 'CO2_uptake_P0.15bar_T298K [mmol/g]',
    'volume [A^3]', 'weight [u]', 'surface_area [m^2/g]',
    'void_fraction', 'void_volume [cm^3/g]',
    ↪ 'largest_free_sphere_diameter [A]',
    'largest_included_sphere_along_free_sphere_path_diameter [A]',
    'largest_included_sphere_diameter [A]',
]

# Load features and labels.
df = pd.read_csv('data/MOFs/all_MOFs_screening_data.csv')
```



```

df.set_index('MOFname', inplace=True)
features = column_names[2:]
target = column_names[1]

reg = RandomForestRegressor(n_jobs=-1)

df_test = df.loc[mof_test]
df_test.replace([np.inf, -np.inf], np.nan, inplace=True)
df_test.dropna(inplace=True)

X_test = df_test.loc[:, features]
y_test = df_test.loc[:, target]

train_sizes = [
    100, 500, 1_000, 2_000, 5_000,
    10_000, 15_000, 20_000, len(mof_train)
]

for size in train_sizes:
    df_train_tmp = df.loc[mof_train[:size]]

    df_train = df_train_tmp.replace([np.inf, -np.inf], np.nan)
    df_train.dropna(inplace=True)

    X_train = df_train.loc[:, features]
    y_train = df_train.loc[:, target]

    reg.fit(X_train, y_train)

    print(size, r2_score(y_test, reg.predict(X_test)))

# Save the trained model.
#with open('rf_model.pkl', 'wb') as fhand:
#     joblib.dump(reg, fhand)

```